

Lecture 20

Transformers

Efficiently modeling sets of objects with attention.

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#2761902**

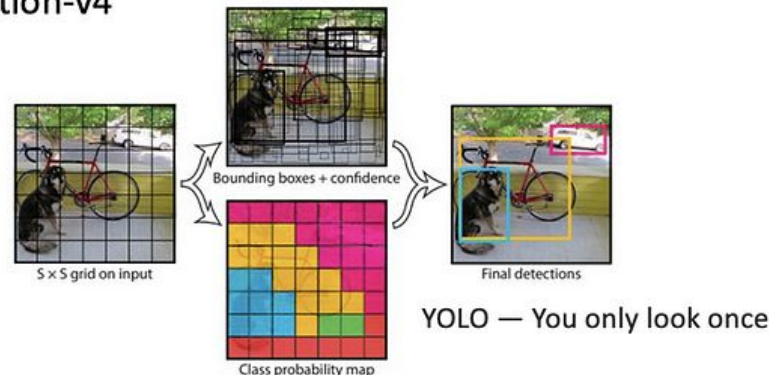
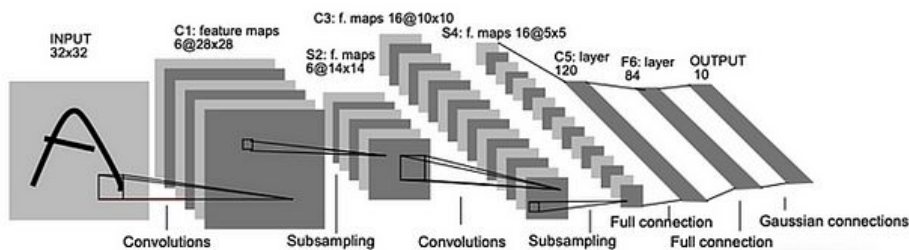
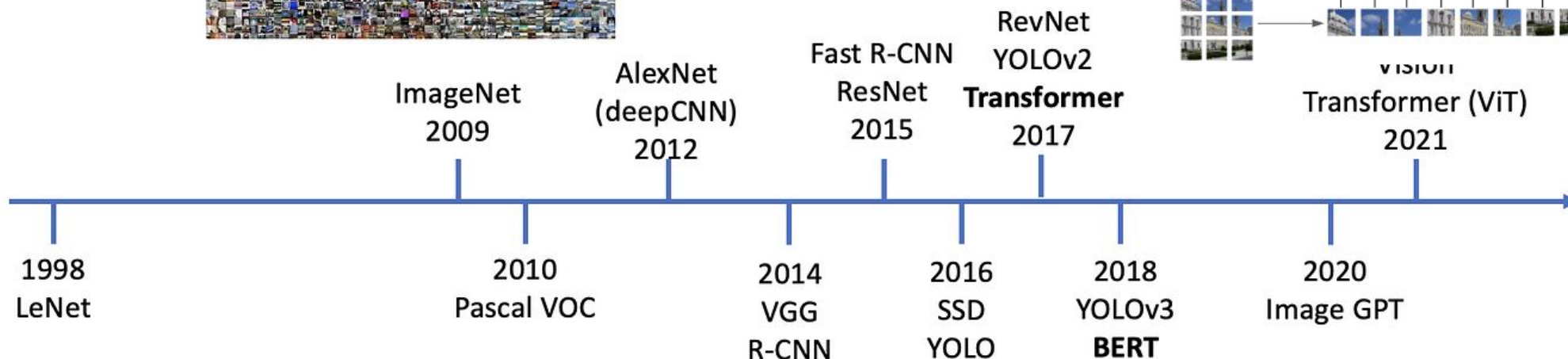
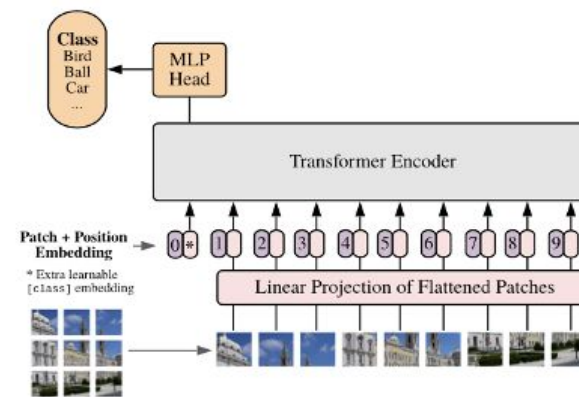
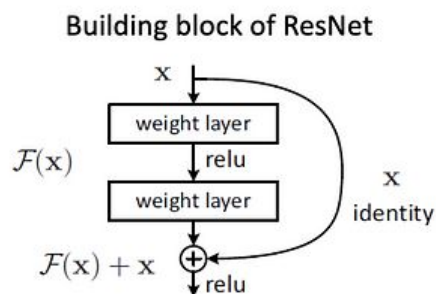
① The Slido app must be installed on every computer you're presenting from

slido

A brief history...



761902





2761902



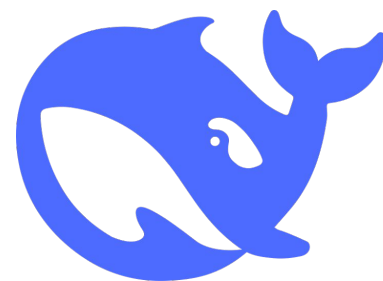
Gemini



Claude



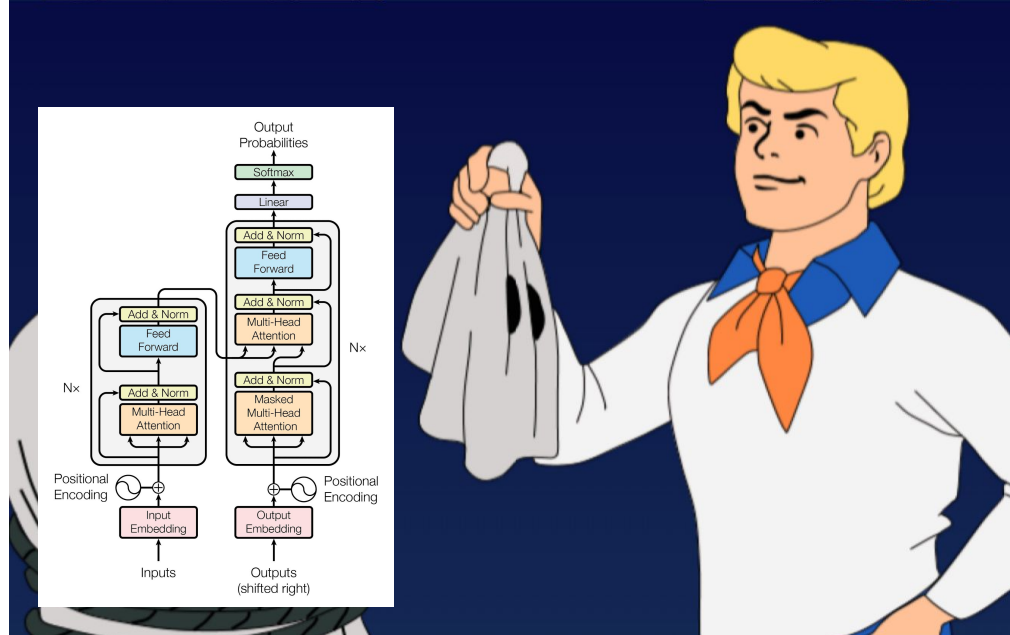
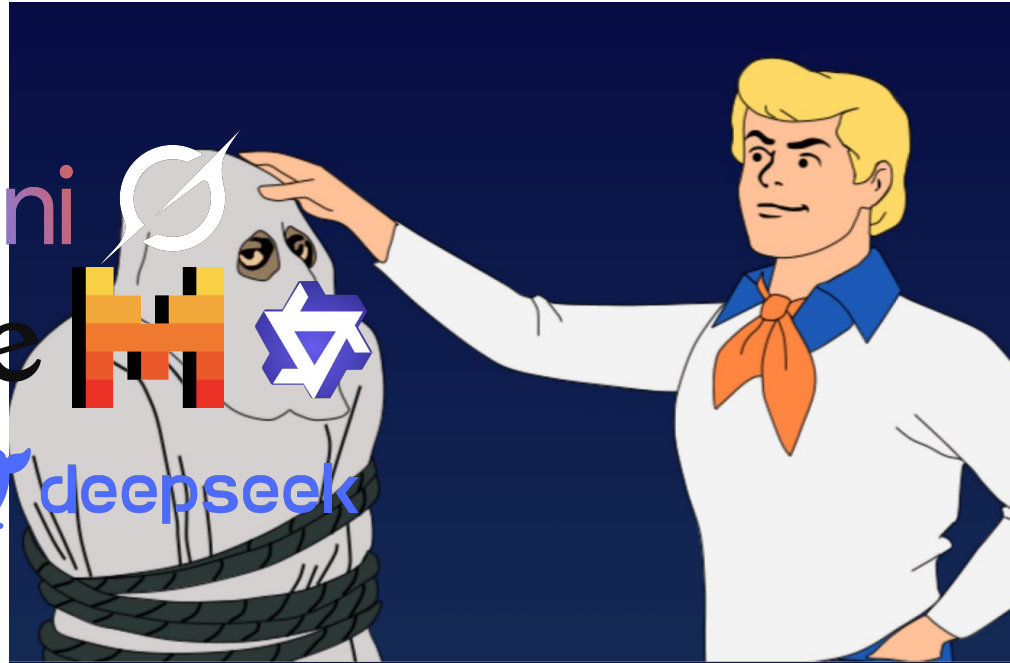
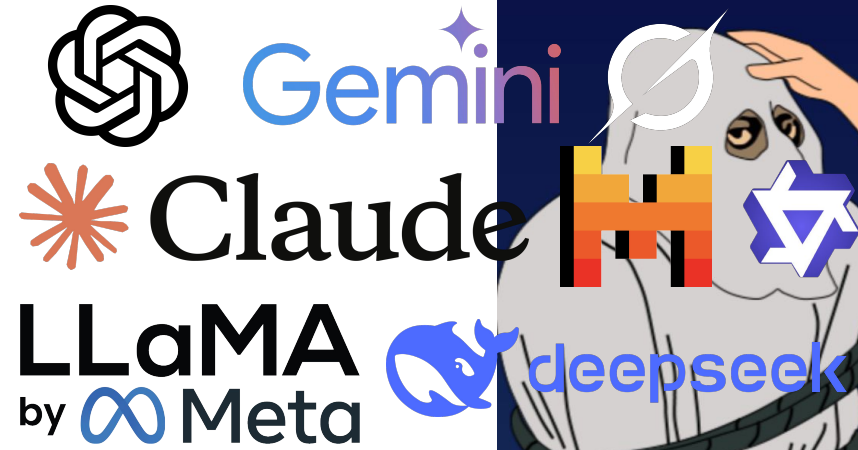
LLaMA
by  Meta



deepseek



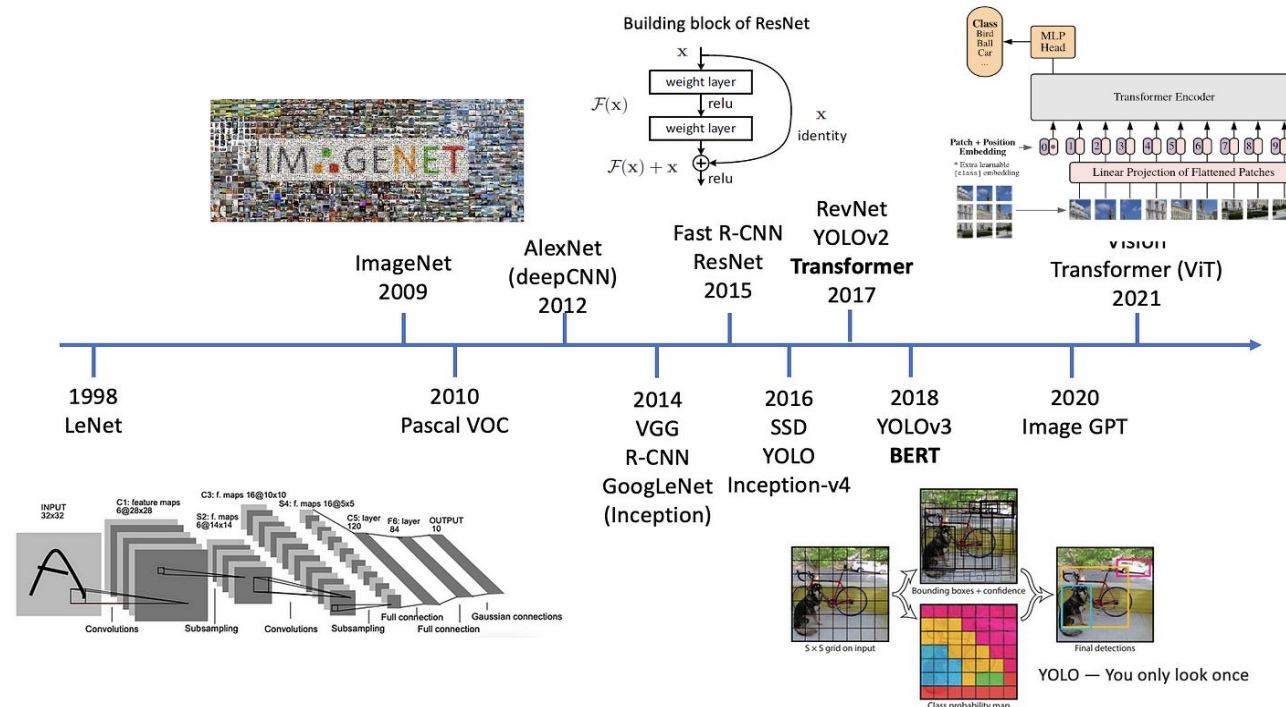
2761902



Transformative Ideas



- **Residual connections** – ubiquitous in modern neural networks!
- **Transformers** – predominant architecture of today's AI/ML era
 - Natural language processing, computer vision, multi-modal models, reinforcement learning, robotics, etc.



Homework 4!



Part 1 – due in 2 weeks (Friday, November 21)

- Paper questions
 - Deep Residual Learning for Image Recognition
 - Attention Is All You Need
- Coding: implement and train CNNs, ResNets, and Transformers

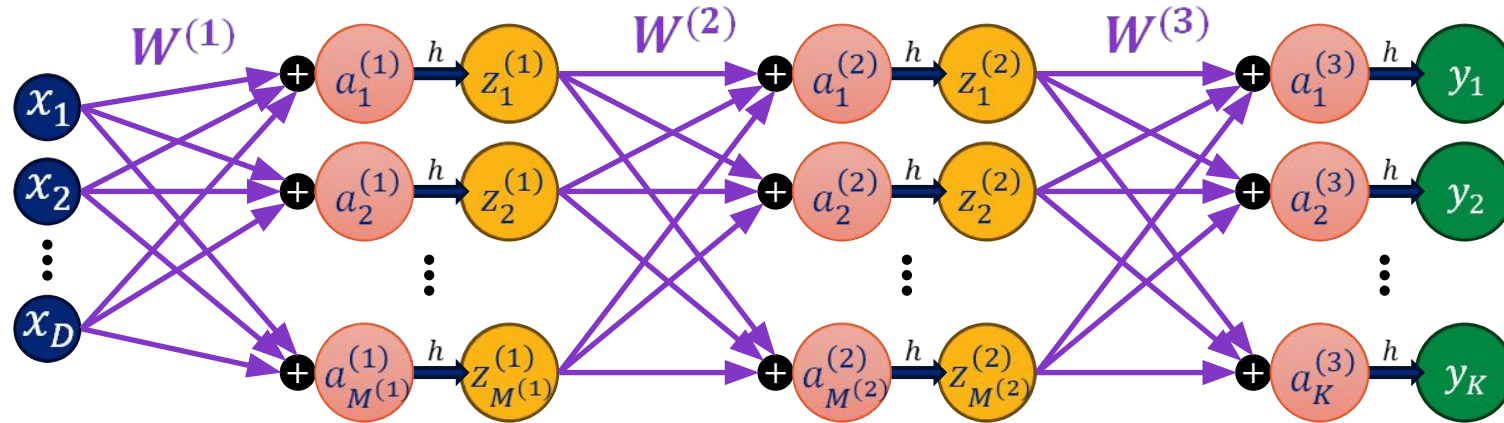
Part 2 – due in 3 weeks (Friday, November 28)

- Coding: DNABERT, ConvNeXt for sound classification
- 2 Kaggle competitions – include your Kaggle username and scores in your PDF for part 2

Important!

- 2 deliverables (PDF and zip file) for the coding portions of both parts
- You can submit up to 5 Kaggle submissions per day

Deep Learning is Feature Transformation



Deep neural networks **transform raw features**, layer by layer

Each layer is a different **representation** of the data

- **Embeddings**: intermediate representations (layers)

Final layers represent the “**abstract concepts**” associated with the prediction task.

MLP Networks

• We started with the “**multi-layer perceptron**”

A **stack** of [Linear Layer + Non-Linearity]

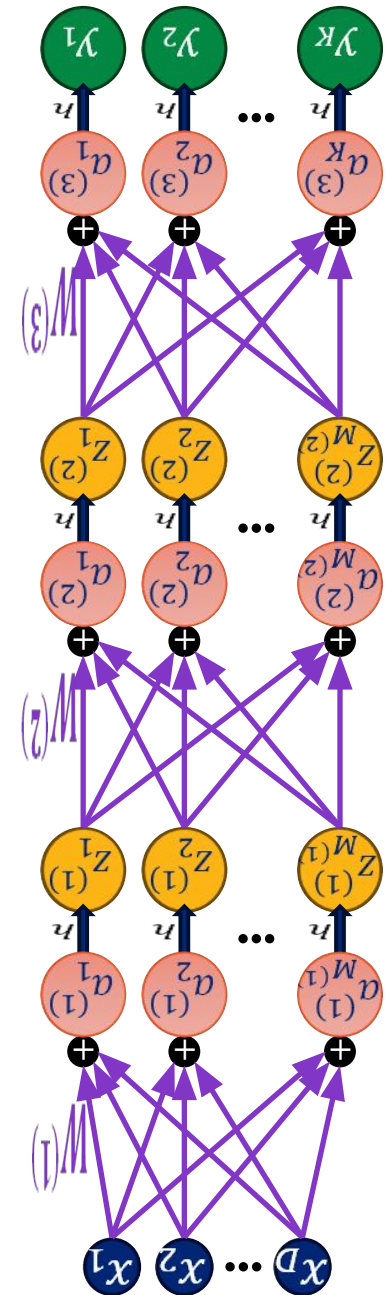
• **Example:** $y = \text{ReLU}(W^{(2)} \text{ReLU}(W^{(1)}x))$

Strengths:

• Highly expressive!

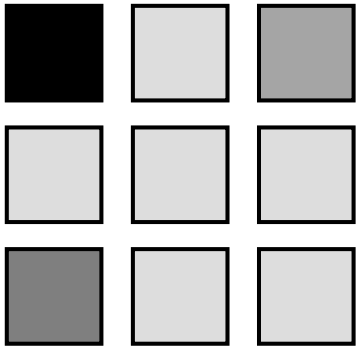
Weaknesses:

- Significant number of parameters!
- Fixed input size.

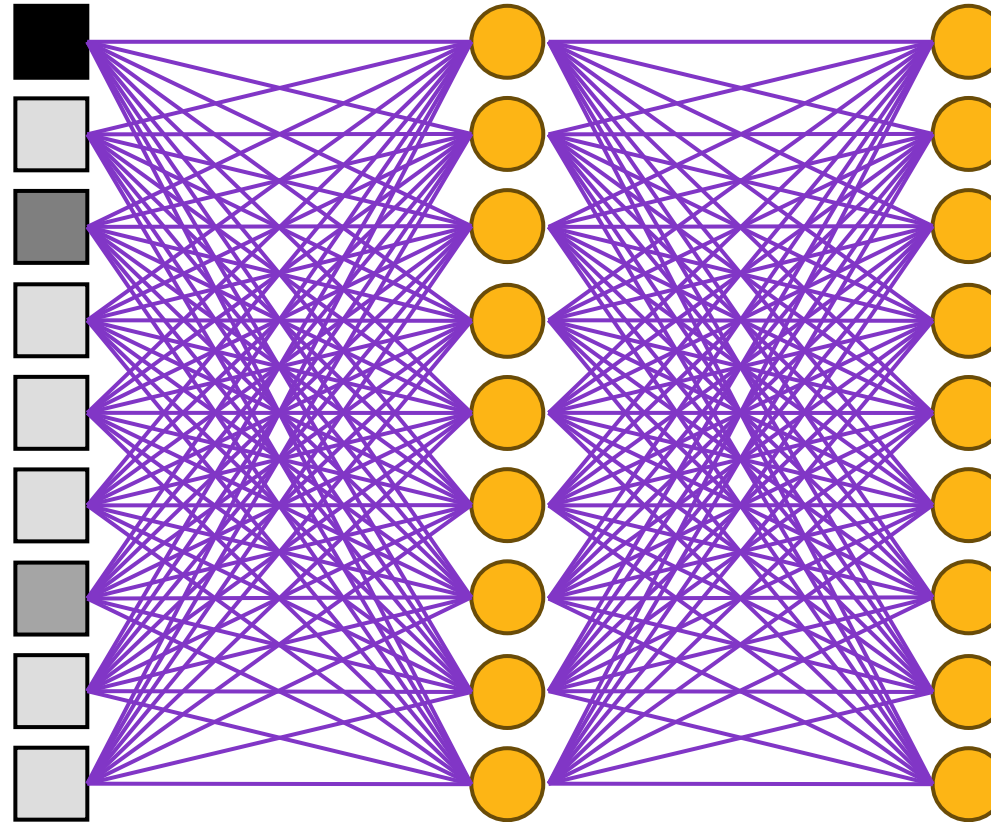


2761902

MLP for Pixels



MLP for Pixels



Separate parameters
for each pixel.

Transformed
Representations
(Embeddings)

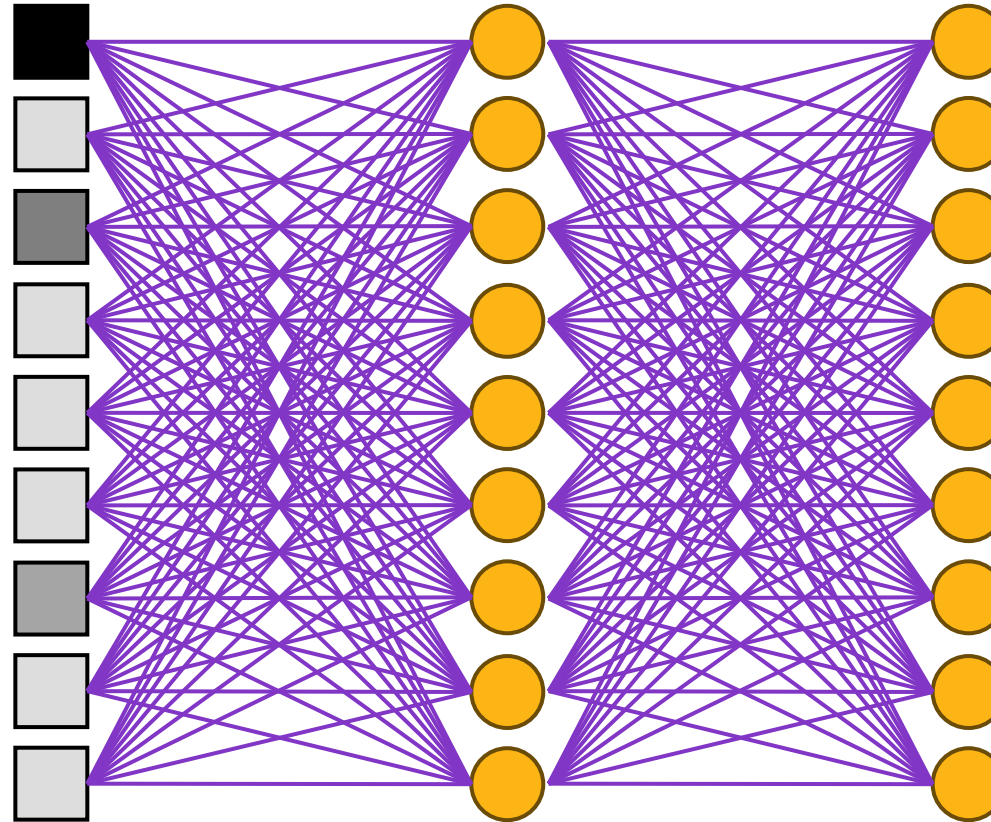
If there are P pixels, how many parameters are in each layer?





If there are P pixels, how many parameters are there in each layer?

MLP for Pixels



Separate parameters
for each pixel.

Transformed
Representations
(Embeddings)

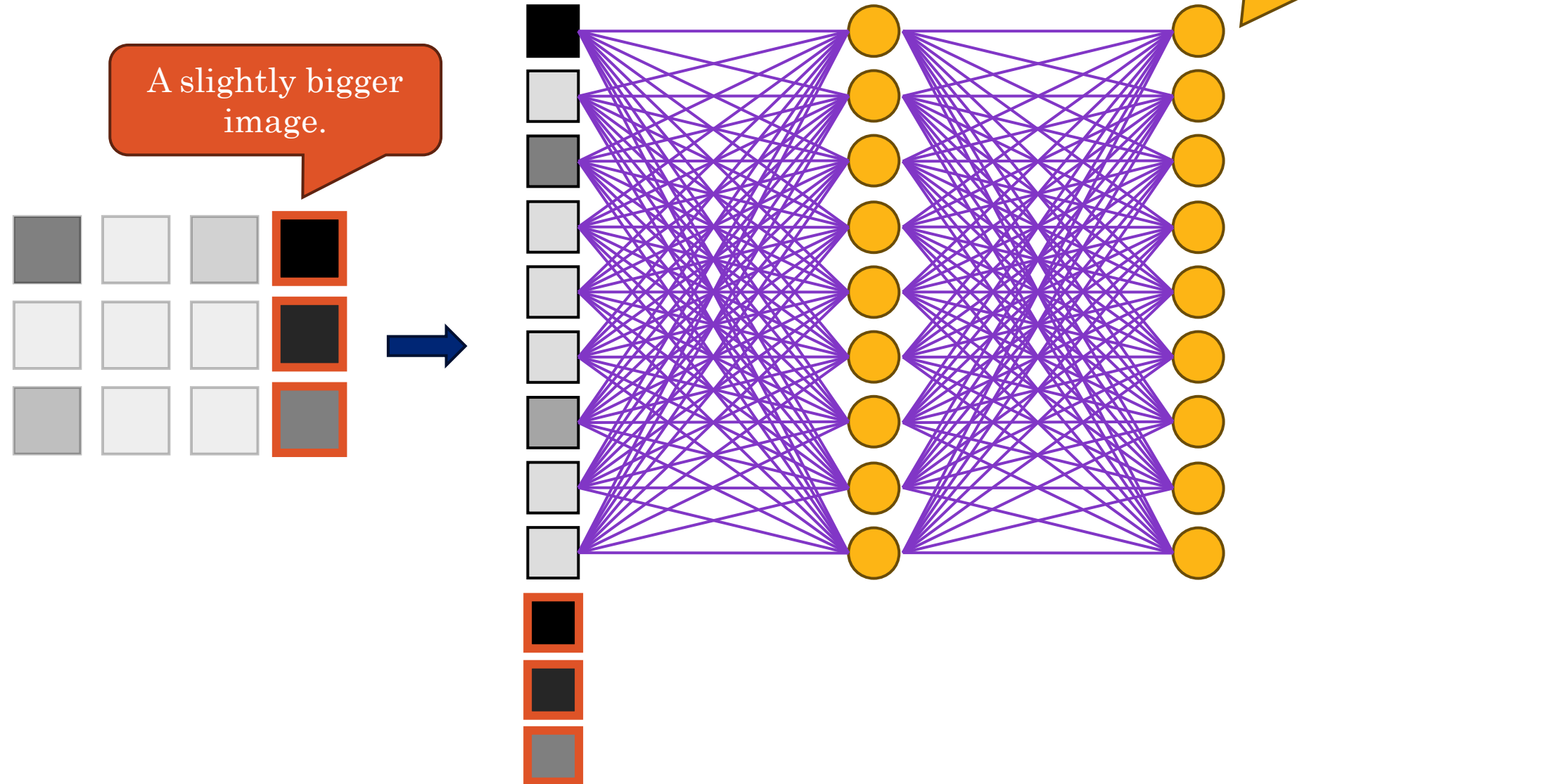
How many Parameters?

If there are P pixels then there are P^2 **weights** per layer.

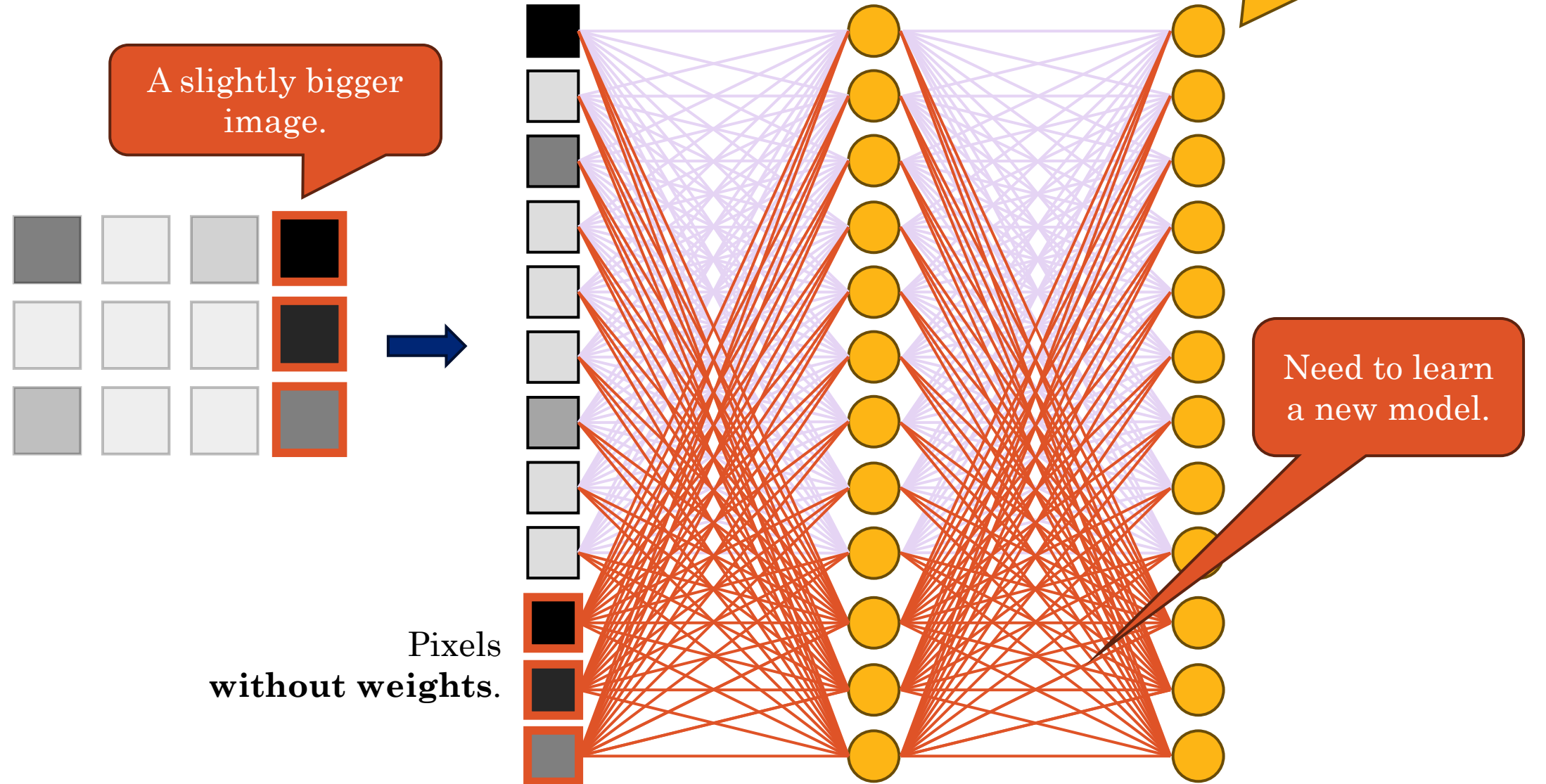
Need to learn a **separate representation** for textures at every location.



MLP for Pixels



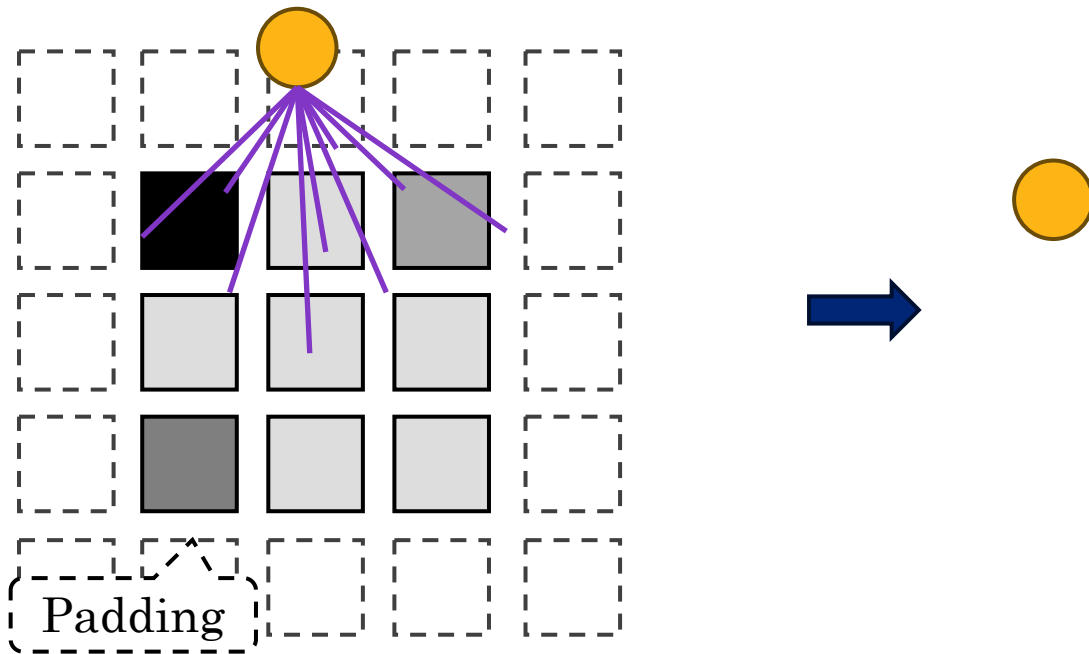
MLP for Pixels



Convolution Networks



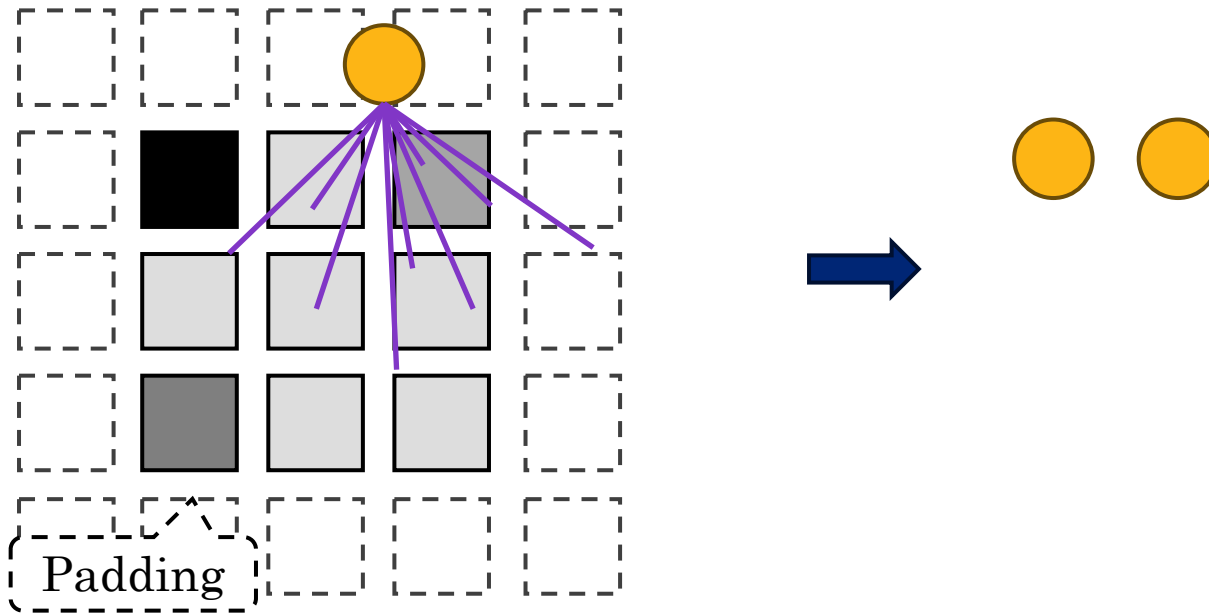
“Slide” (convolve) a neuron across the input.



Convolution Networks



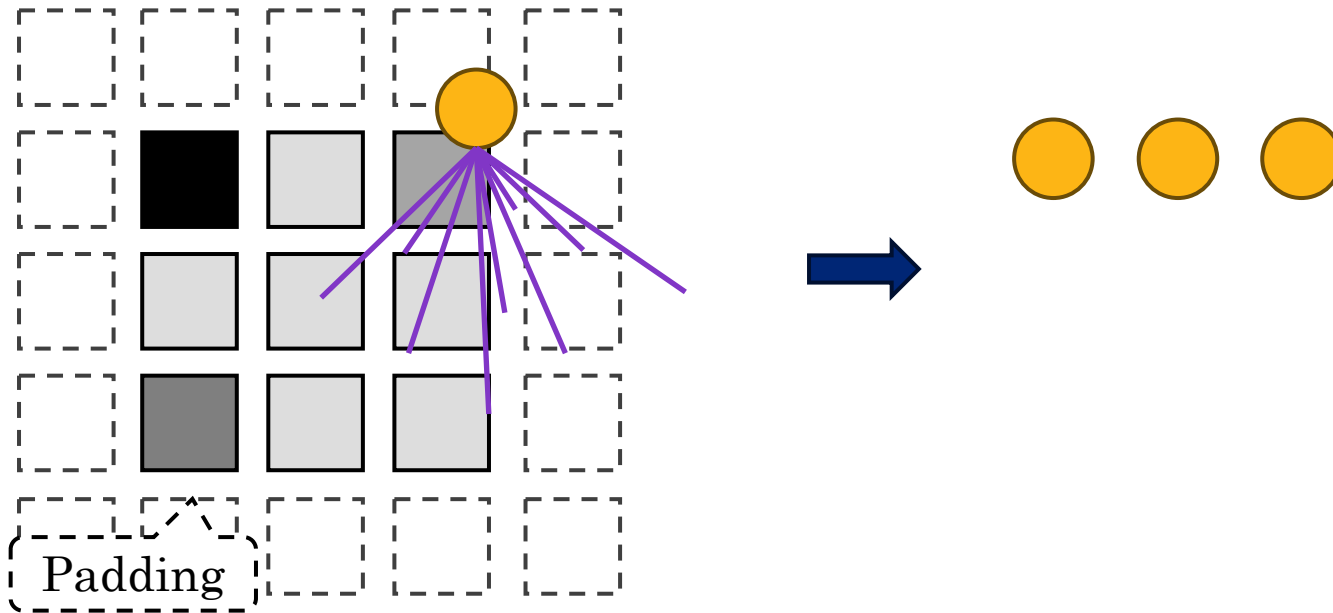
“Slide” (convolve) a neuron across the input.



Convolution Networks



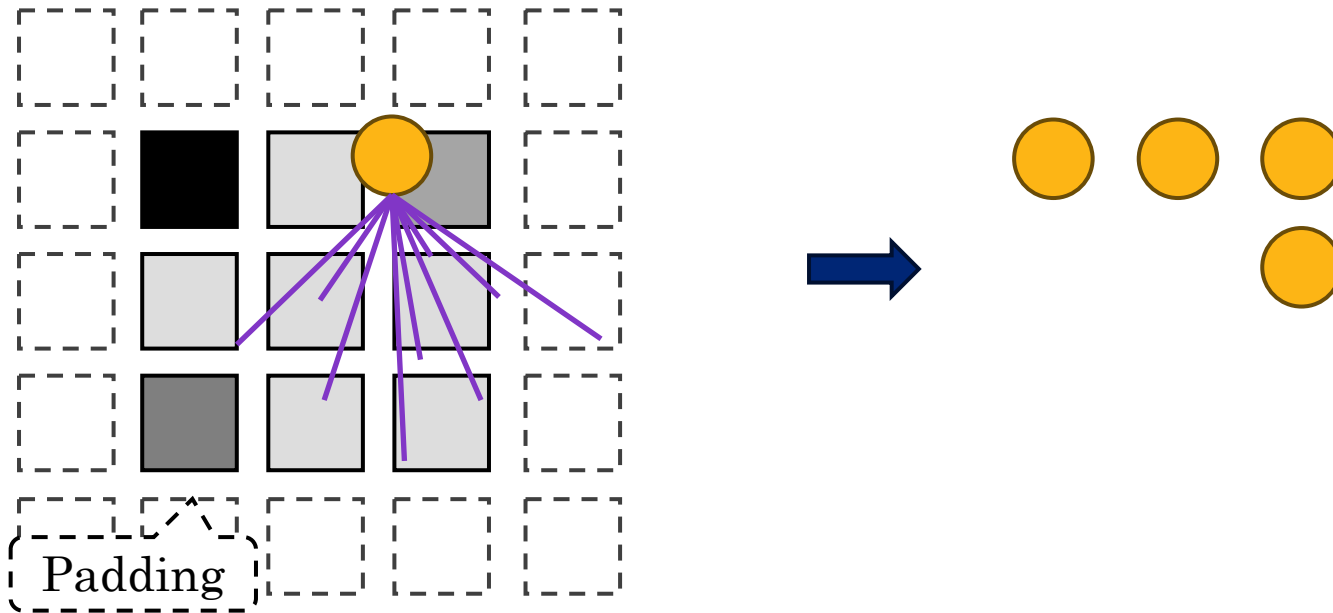
“Slide” (convolve) a neuron across the input.



Convolution Networks



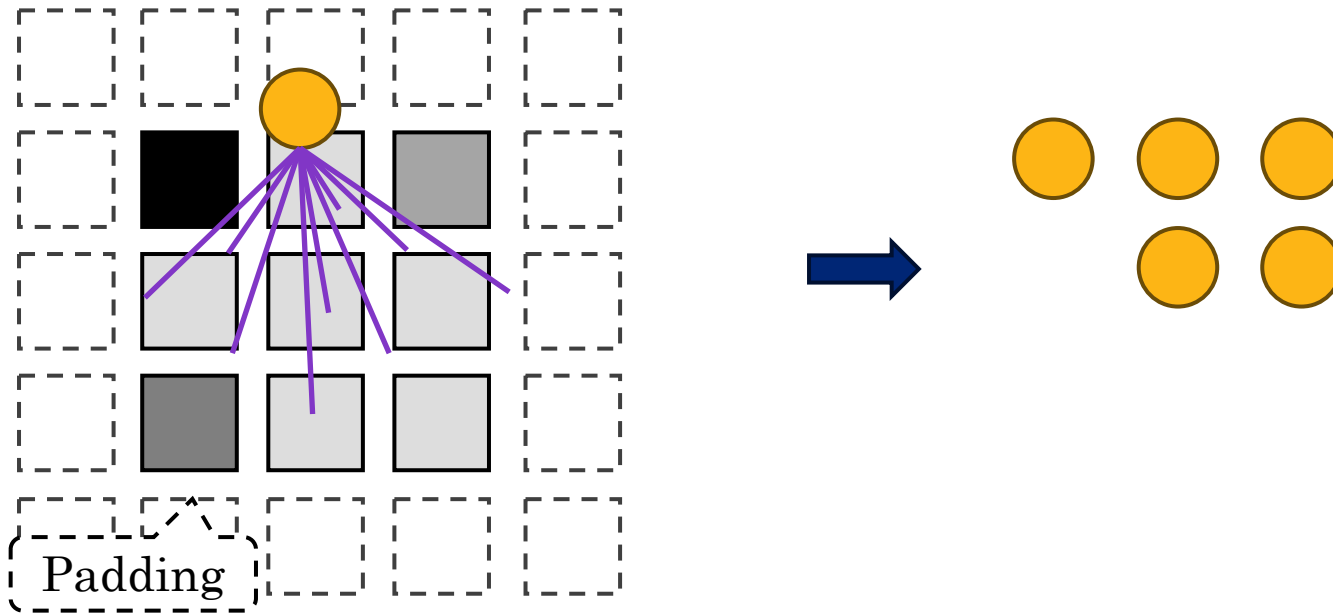
“Slide” (convolve) a neuron across the input.



Convolution Networks



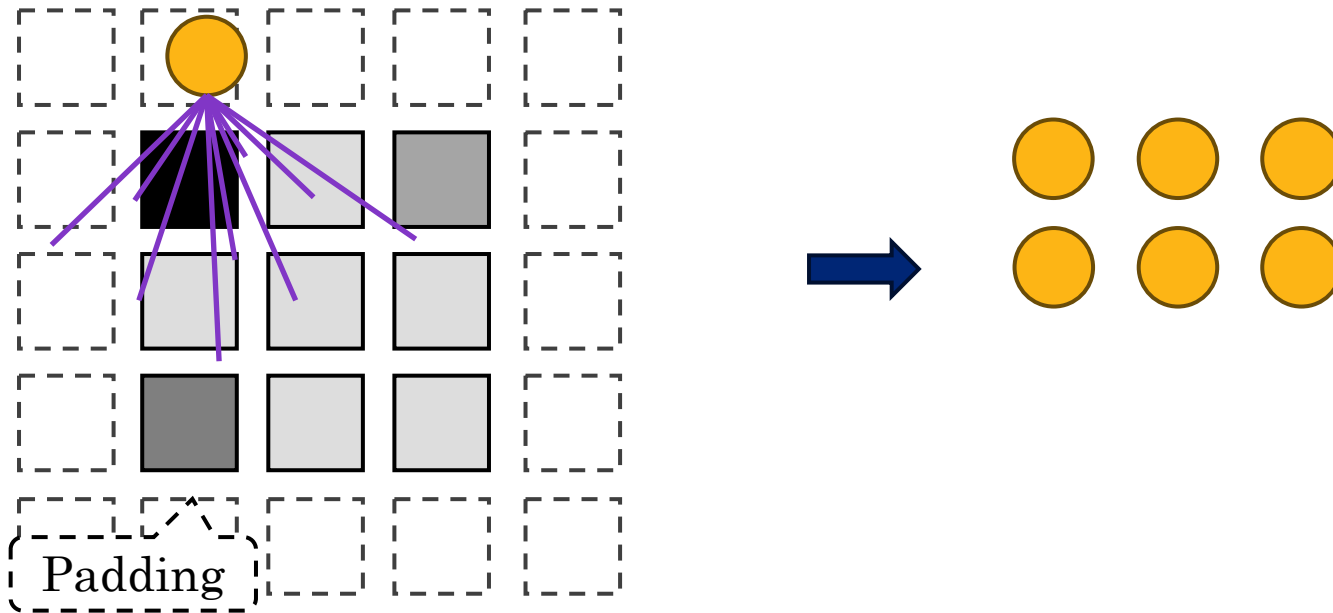
“Slide” (convolve) a neuron across the input.



Convolution Networks



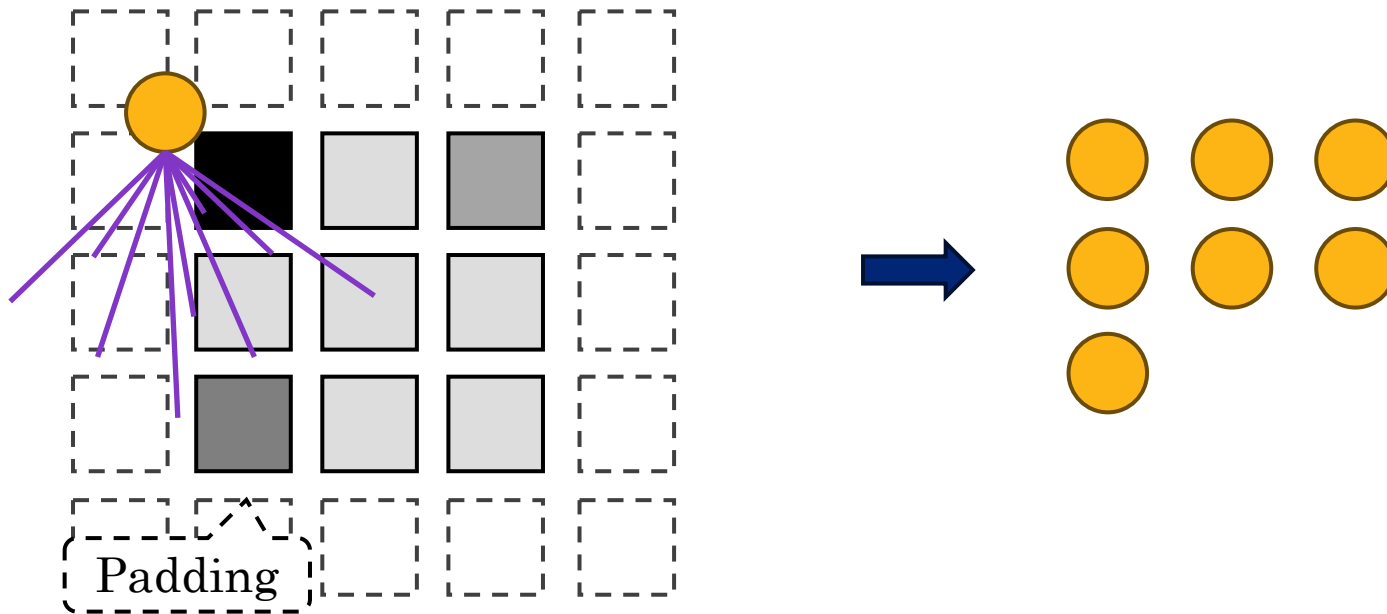
“Slide” (convolve) a neuron across the input.



Convolution Networks



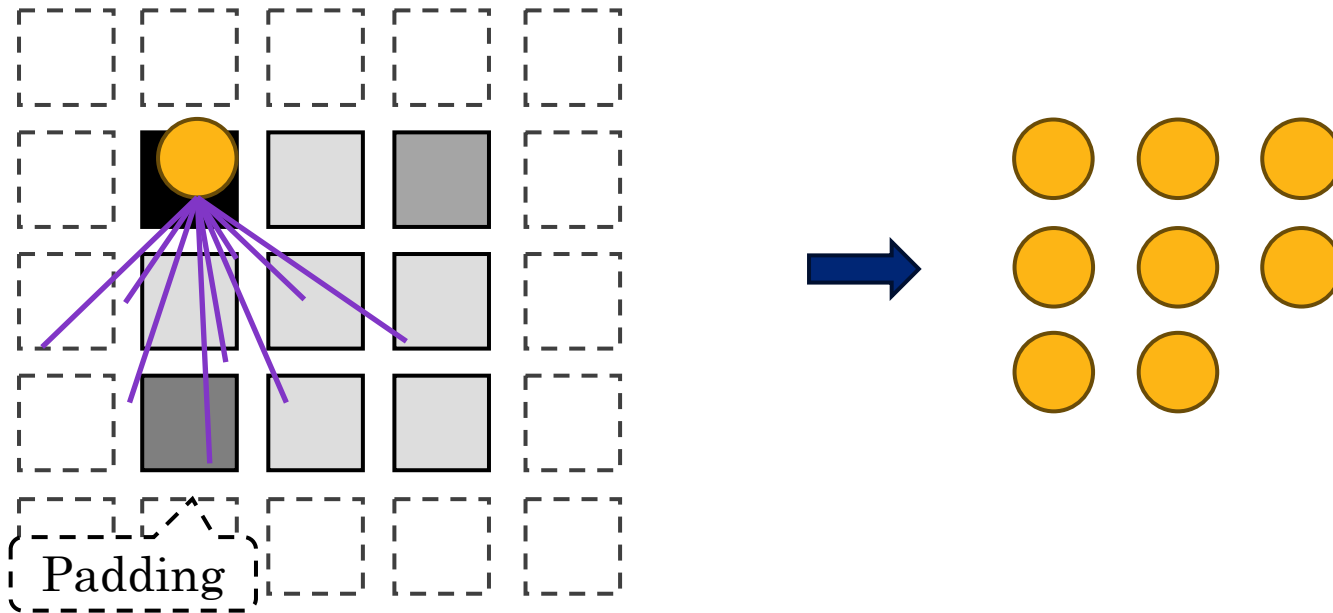
“Slide” (convolve) a neuron across the input.



Convolution Networks



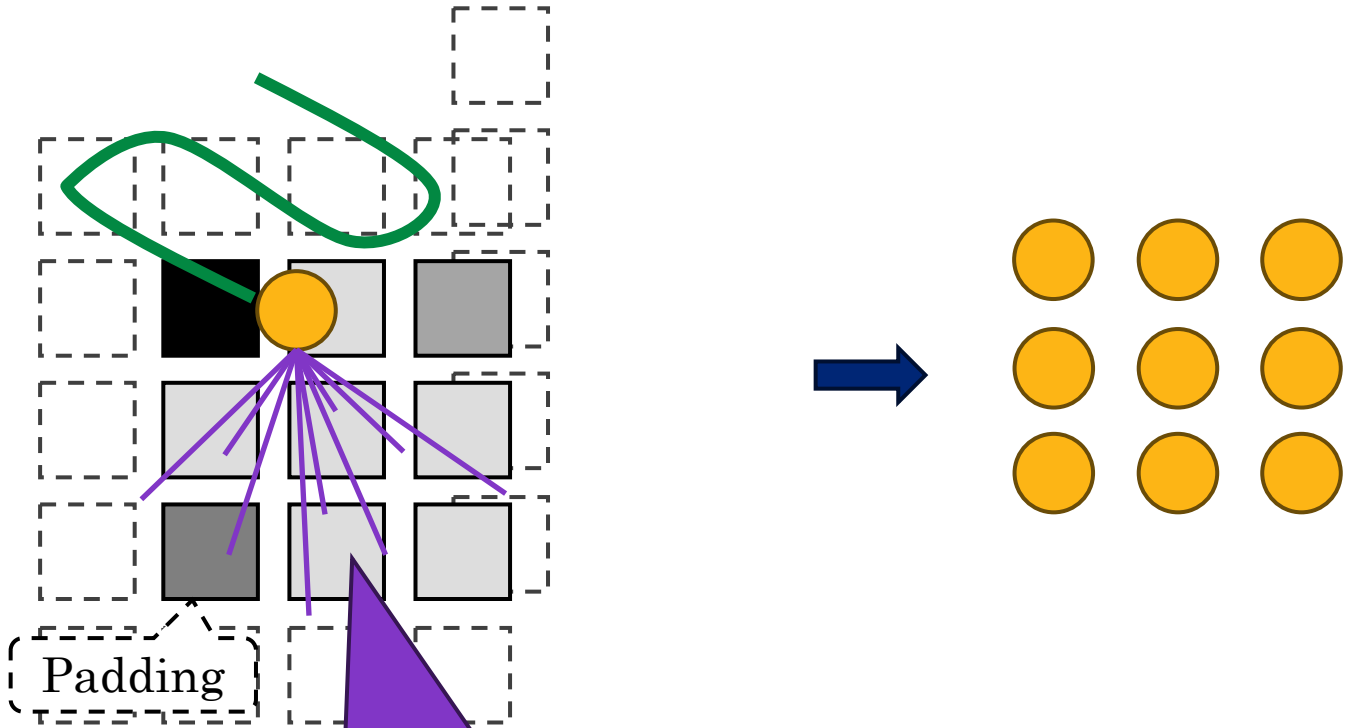
“Slide” (convolve) a neuron across the input.



Convolution Networks



“Slide” (convolve) a neuron across the input.

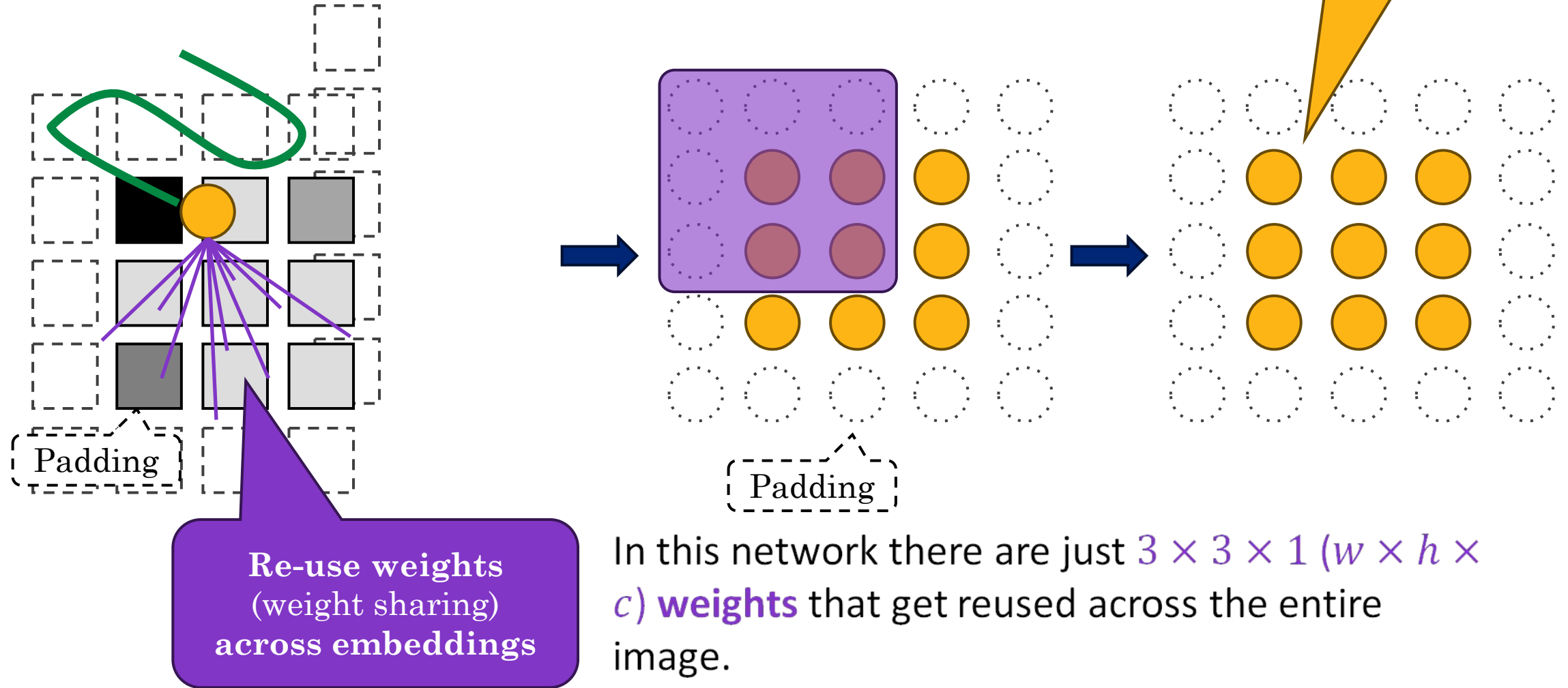


Re-use weights
(weight sharing)
across embeddings

In this network there are just $3 \times 3 \times 1$ ($w \times h \times c$) **weights** that get reused across the entire image.

Convolution Networks

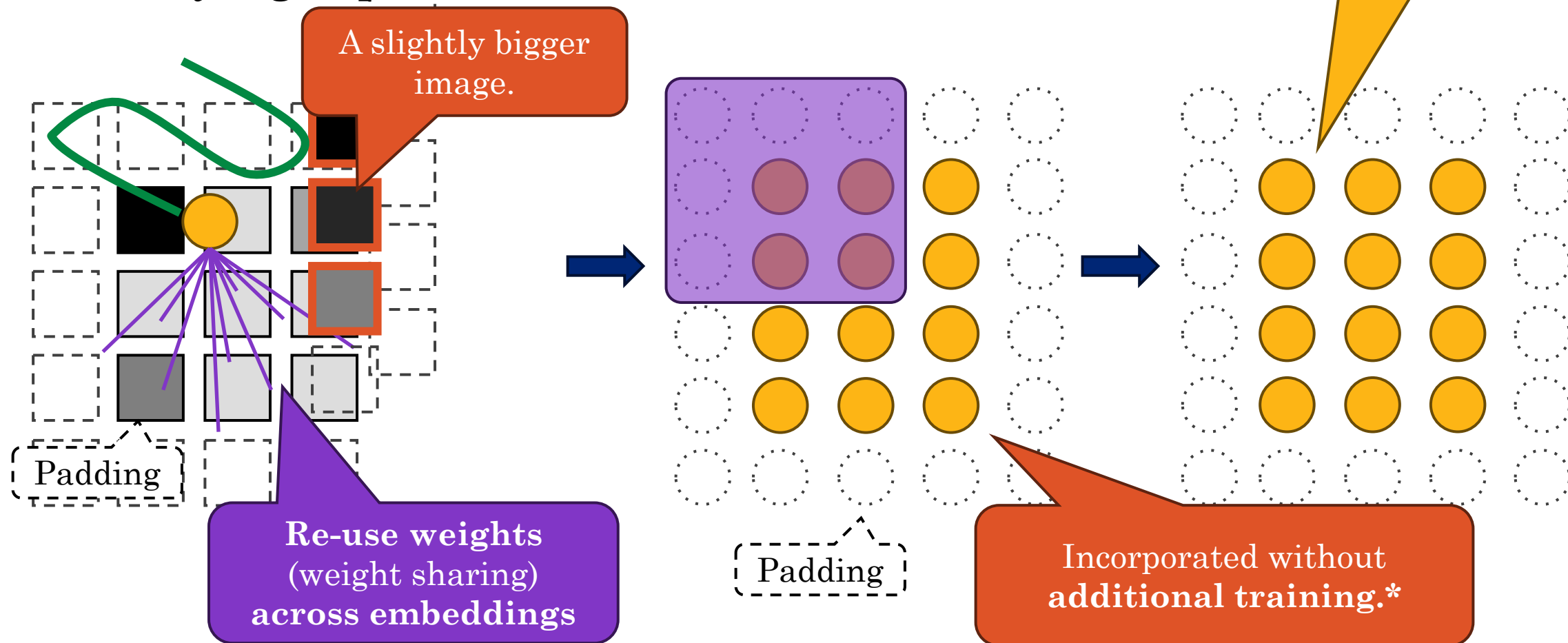
Provide deep transformed representations.



2761902

Convolution Networks

Enable varying input sizes.



*Significant changes in image resolution may required deeper models and additional training.



2761902

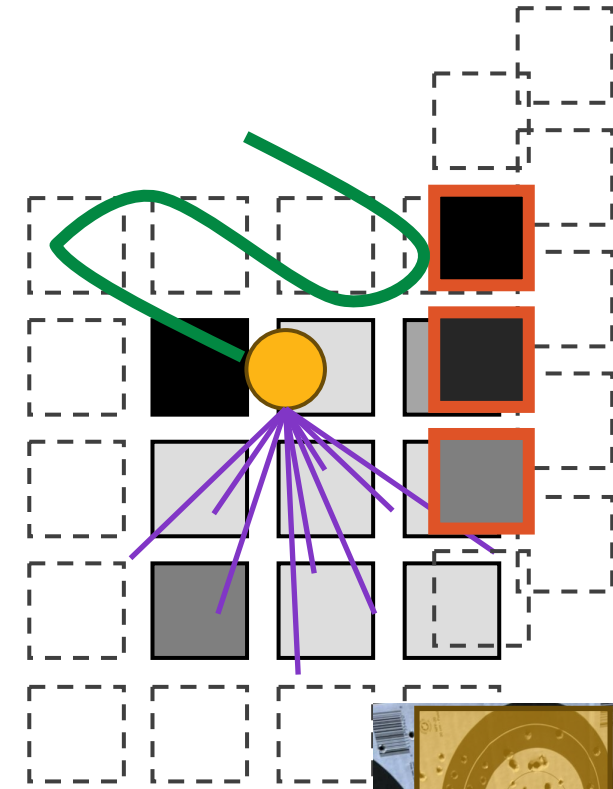
Convolution Networks

“Slide” (convolve) a neuron across the input.

Strengths:

- Parameter efficient
- Encode **spatial invariance** inductive bias
- Enable **varying input dimensions**
 - At least spatially (not channel dimension)

Weaknesses?



2761902



What is in this image?

Broader context
is often required to
understand a part.



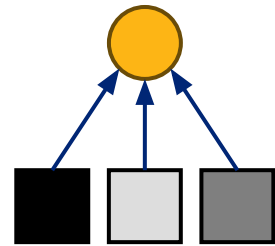
2761902

Firefighter
spiderman

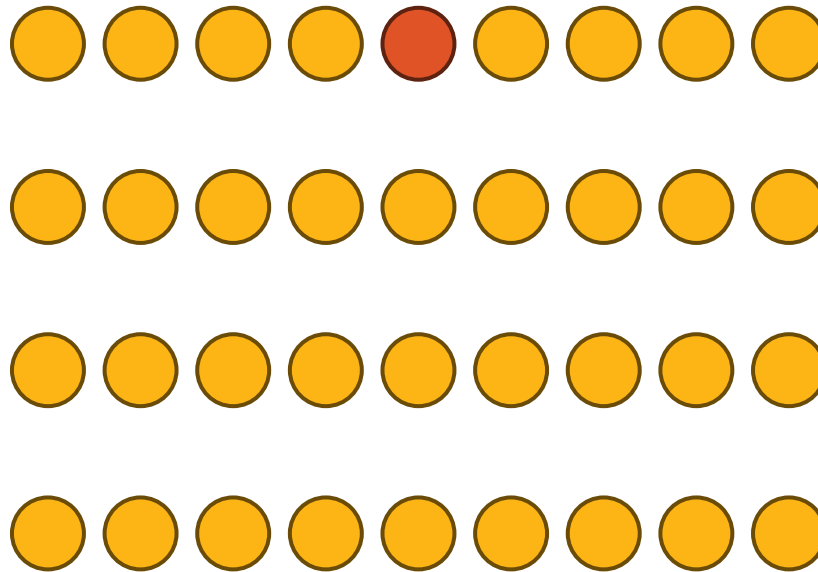
Convolution Receptive Field



What is the width of the **receptive field** (in pixels) of the **top center activation** assuming a width 3 convolution **on all layers**



Width 3
Convolution



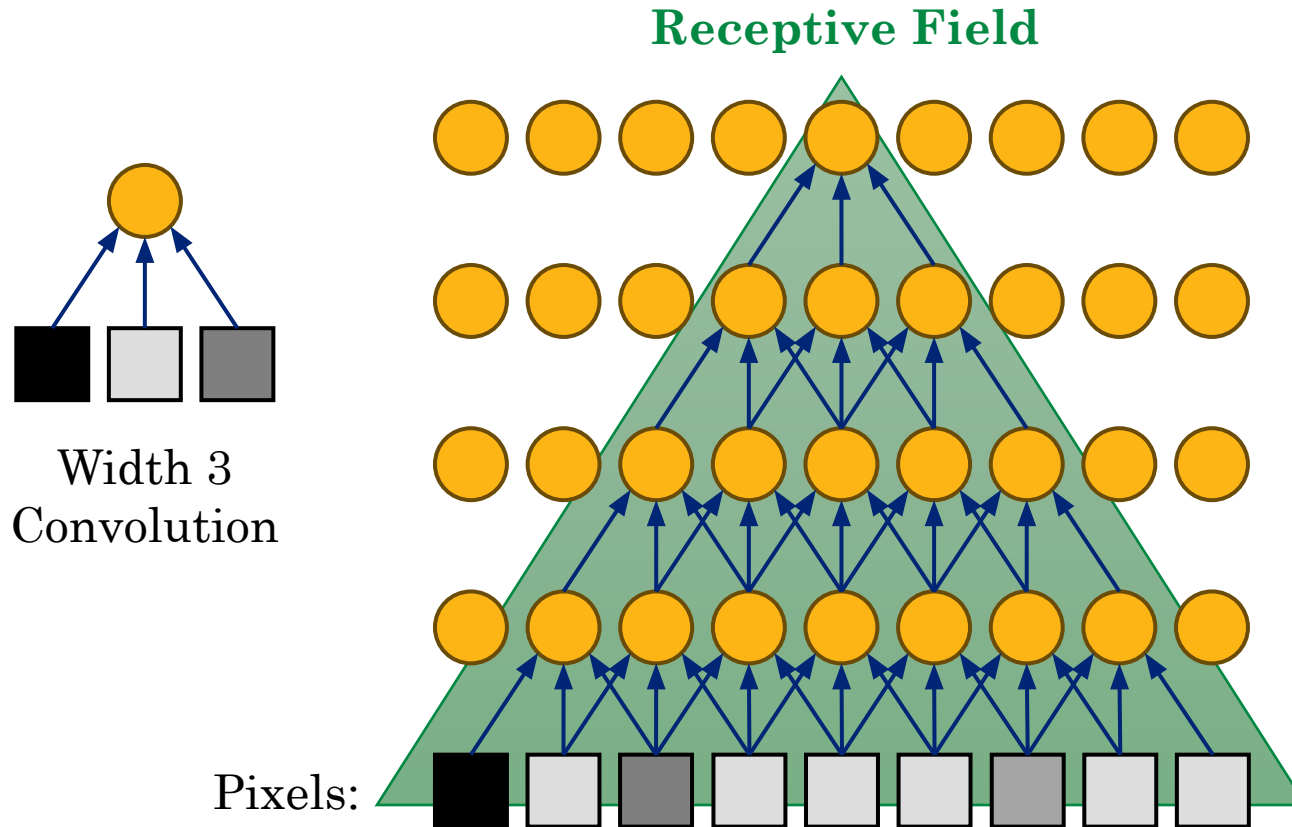


What is the width of the receptive field (in pixels) of the top center activation assuming a width 3 convolution on all layers

Convolution Receptive Field



Higher layers in the network see more context.



Convolution Networks

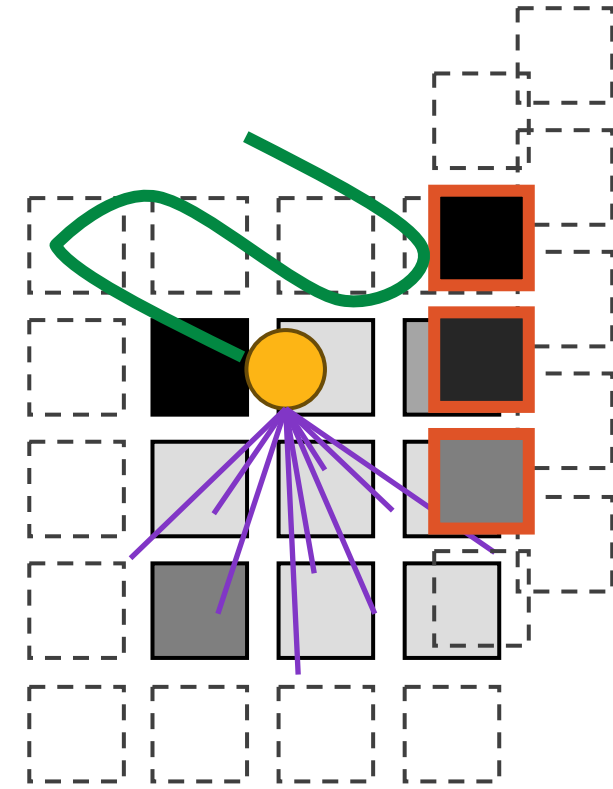
“Slide” (convolve) a neuron across the input.

Strengths:

- Parameter **efficient**
- Encode **spatial invariance** inductive bias
- Enable **varying input dimensions**
 - At least spatially (not channel dimension)

Weaknesses:

- **Builds local representations** and only **top layers** of the network have “full” context (can see everything...).



Soln: Transformer Architecture

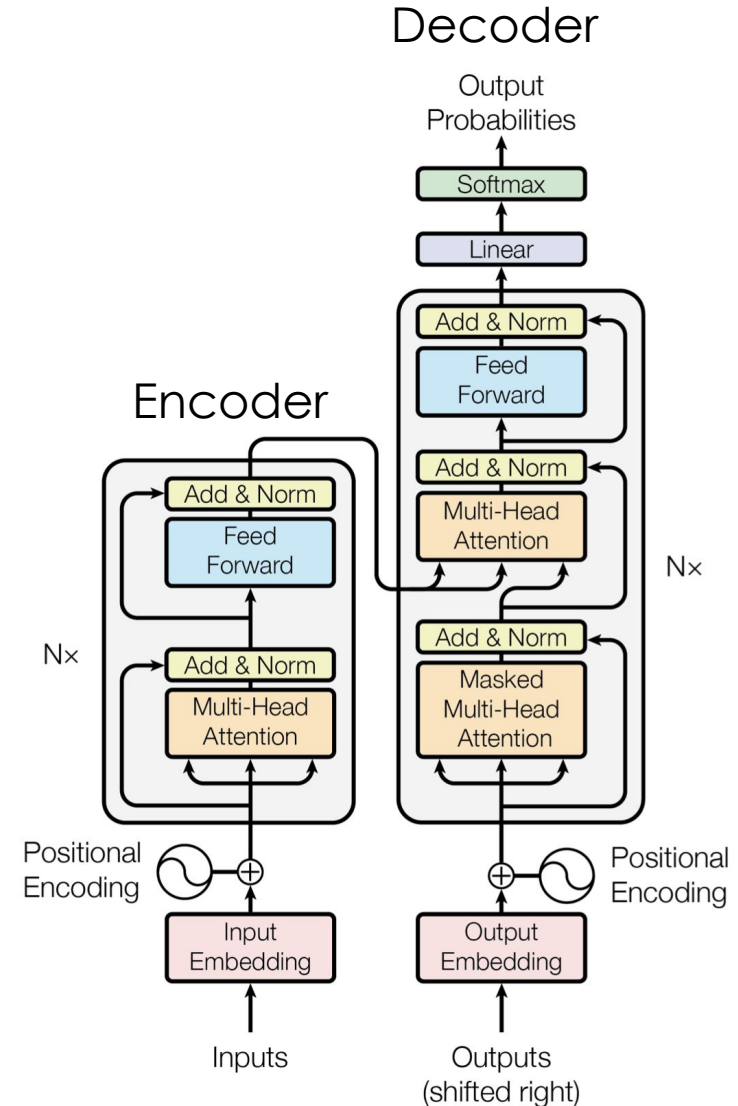


The basic building block of **state-of-the-art** (SoS) **neural networks** for **most domains** (e.g., ChatGPT, Speech Recognition, Image and Video Generation)

Originally introduced for **machine translation**

- **Attention is All You Need** (homework!)
- “Somewhat” difficult to follow...

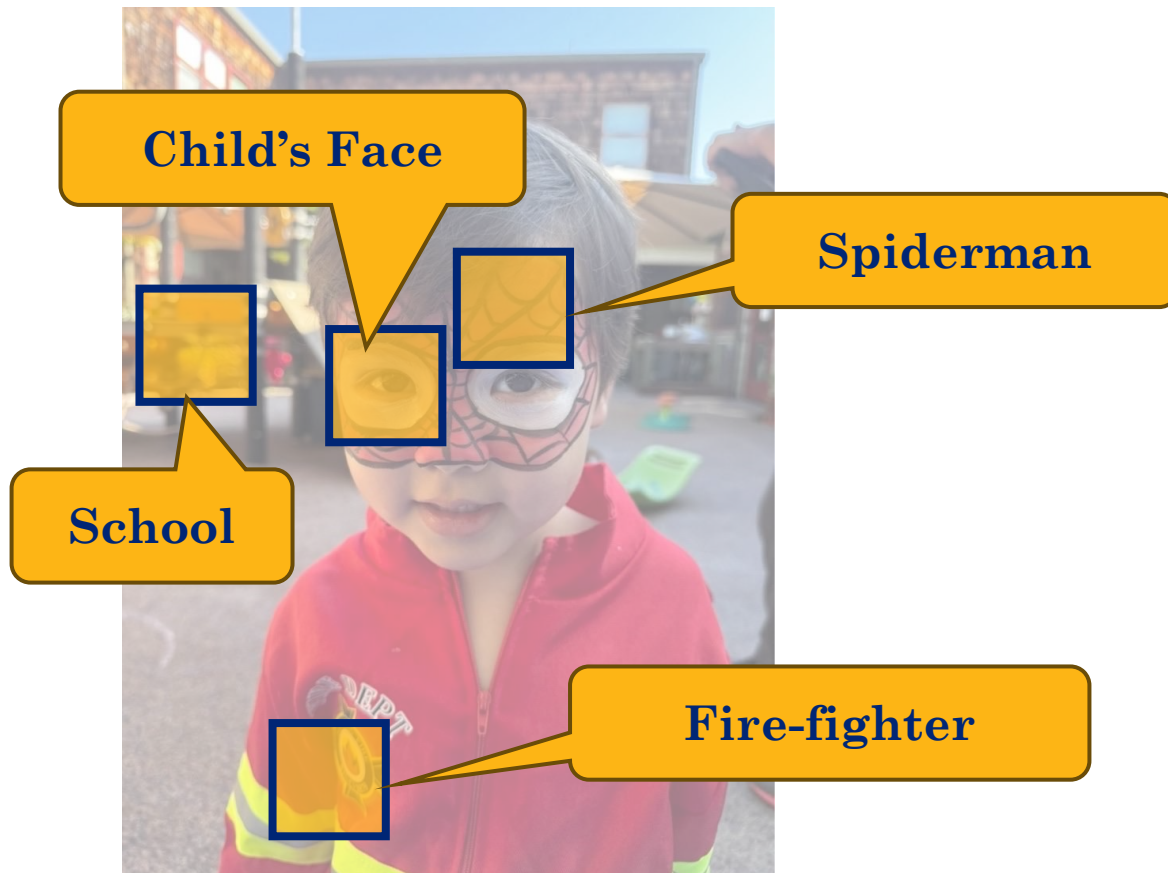
We will try to build the Transformer from **first principles** spanning **language** and **visual modeling**



Need to Include Context in Representations



Computer Vision



Language Modeling

*I swam across the river to get to the other **bank**.*

*I walked across the road to get cash from the other **bank**.*

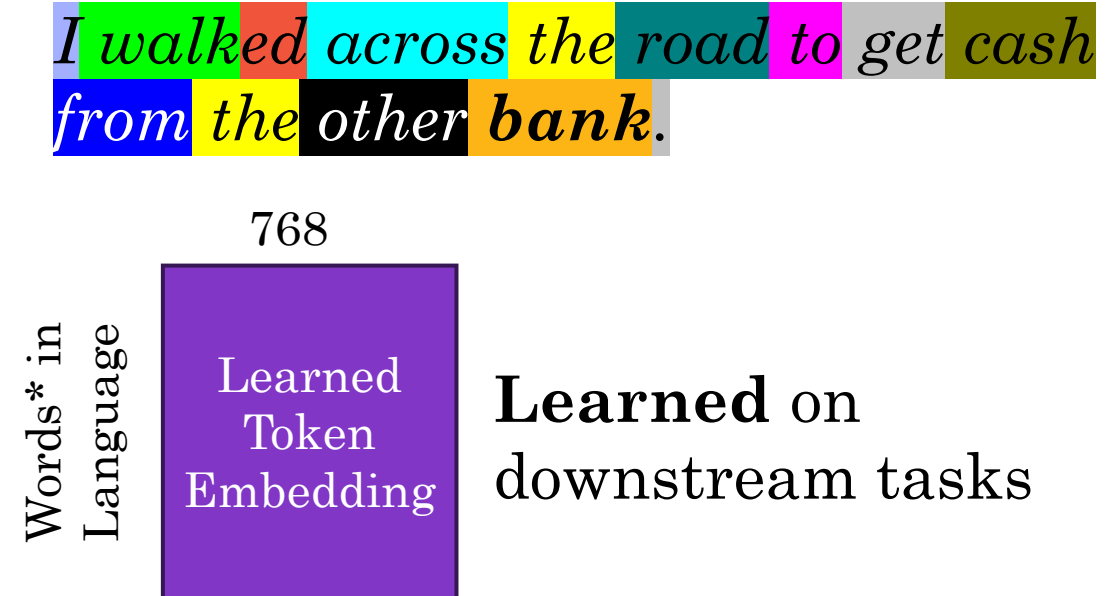
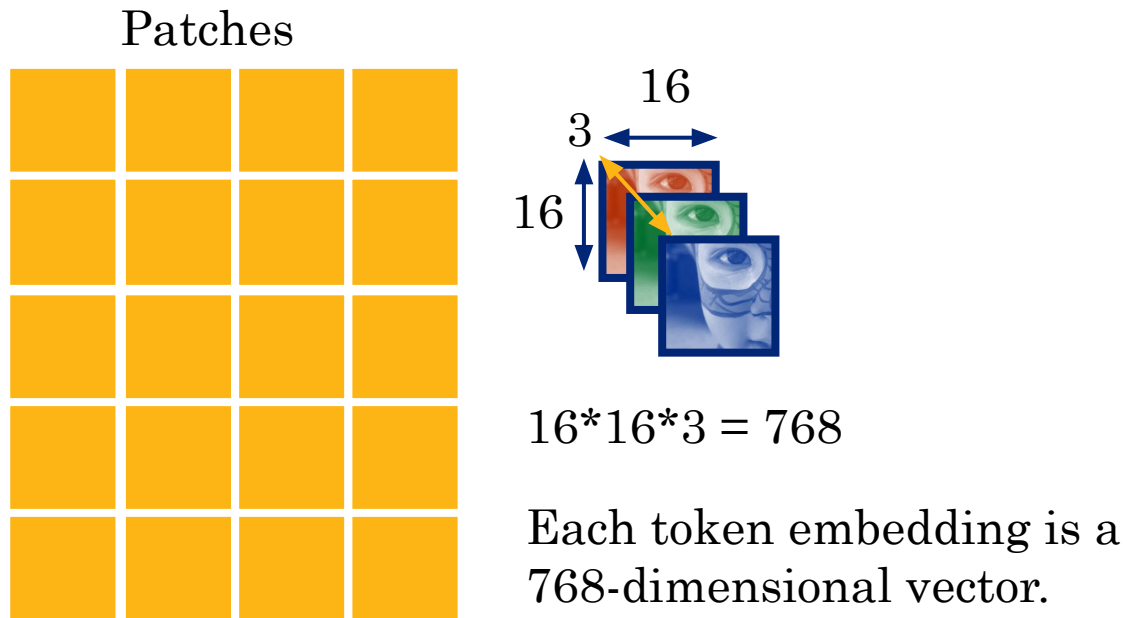
The **meaning of bank** depends on the words (context) around it.



Starting with Token Embeddings

Token Embeddings: The Transformer model takes as input a set of N token embeddings vectors in $\mathbb{R}^D$ representing the basic building blocks of an image or text.

- **Example:** Patches of images or words*



*Typically, model word parts not words (more later).



Basic Setup of a Transformer Layer

Input: A matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ of N token embeddings vectors in $\mathbb{R}^D$

Output: A matrix $\mathbf{Y} \in \mathbb{R}^{N \times D}$ of N token embeddings vectors in $\mathbb{R}^D$

Step 1: Start by **extracting the “context”** from each of the patches in isolation using a linear layer:

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

where $\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$ is a learned matrix.

- We will determine D_v later but you can think of this as D for now.
- **Note** we will “imagine” that $\mathbf{V}$ encodes the “context” but SGD will determine the “true meaning” of $\mathbf{V}$.



Incorporating the Context of other Tokens

Focusing on the n^{th} token $\mathbf{Y}_n$ how could we incorporate the context of all the other tokens?

Proposal – Average Pooling:

$$\mathbf{Y}_n = \mathbf{X}_n + \frac{1}{N} \sum_{i=1}^N \mathbf{v}_i$$

- Combines all the “extracted context”
- No additional parameters
- Invariant to number of tokens?
 - Yes! Averaging more tokens should behave similarly.
- But...

Symbols:

$$\begin{aligned}\mathbf{X} &\in \mathbb{R}^{N \times D} \\ \mathbf{Y} &\in \mathbb{R}^{N \times D} \\ \mathbf{W}^{(v)} &\in \mathbb{R}^{D \times D_v} \\ \mathbf{V} &= \mathbf{X} \mathbf{W}^{(v)}\end{aligned}$$



What are some weaknesses of average pooling to incorporate the context of other tokens?



Not all Context is Relevant

Averaging context **incorrectly** treats all context as **equally important** to the context of the n^{th} token $\mathbf{Y}_n$:

*I **swam** across the **river** to get to the other **bank**.*
Most important
Important

Some context could be harmful:

*The **food** was **good**, not **bad** at all.*
Harmful? *
Important

*If “bad” becomes **transformed** by “not” (context) then it might be helpful.



Attention: Weighted Averaging of Context

Proposal: Focus pooling on **relevant tokens** -- **attention**

$$\mathbf{Y}_n = \mathbf{X}_n + \sum_{i=1}^N \alpha_{ni} \mathbf{V}_i$$

where the **attention weights** $\alpha_{ni} \geq 0$ and $\sum_{i=1}^N \alpha_{ni} = 1$.

- A **weighted average** of the context of other tokens

Goals: assign **large weight** to **relevant tokens** and smaller or near zero weight to **irrelevant tokens**

Proposal: Leverage **inner-product search** from information retrieval

Symbols:

$\mathbf{X} \in \mathbb{R}^{N \times D}$

$\mathbf{Y} \in \mathbb{R}^{N \times D}$

$\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$

$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$



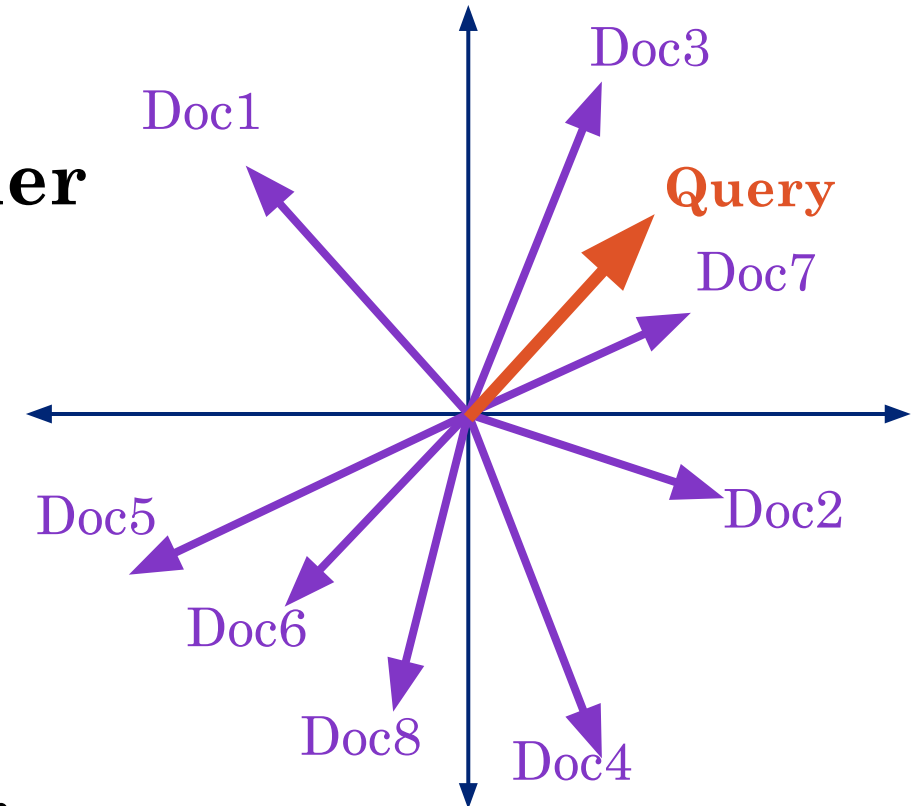
Classic Inner-Product Search for IR

Inner-Product Search is a basic **information retrieval (IR)** method to rank document relevance by:

1. Assigning **vector “key”** to **each document**.
2. Constructing a **query vector**.
3. Returns documents in **descending order** of the inner-product.

Example: Doc3, Doc7, Doc2, Doc1, ...

The IR community has developed numerous techniques to construct these vectors but here we will use **linear layers**.





Constructing Keys and Queries

We construct the “**key**” vectors and “**query**” vectors for each token using a linear layer:

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

where $\mathbf{W}^{(k)}, \mathbf{W}^{(q)} \in \mathbb{R}^{D \times D_k}$ are parameters and the size of the key vector D_k is typically chosen to be smaller than or equal to D .

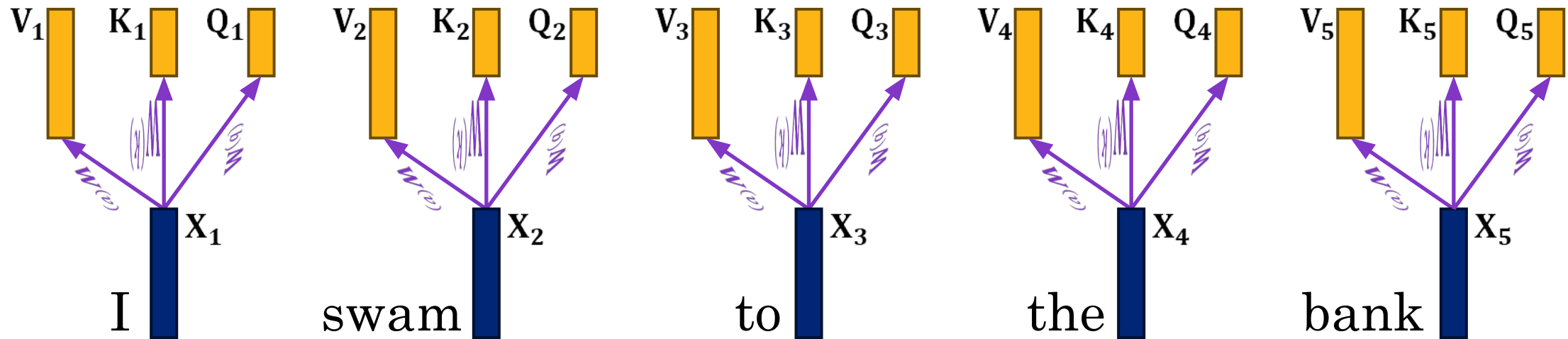
Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{Y} \in \mathbb{R}^{N \times D}$$

$$\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$$

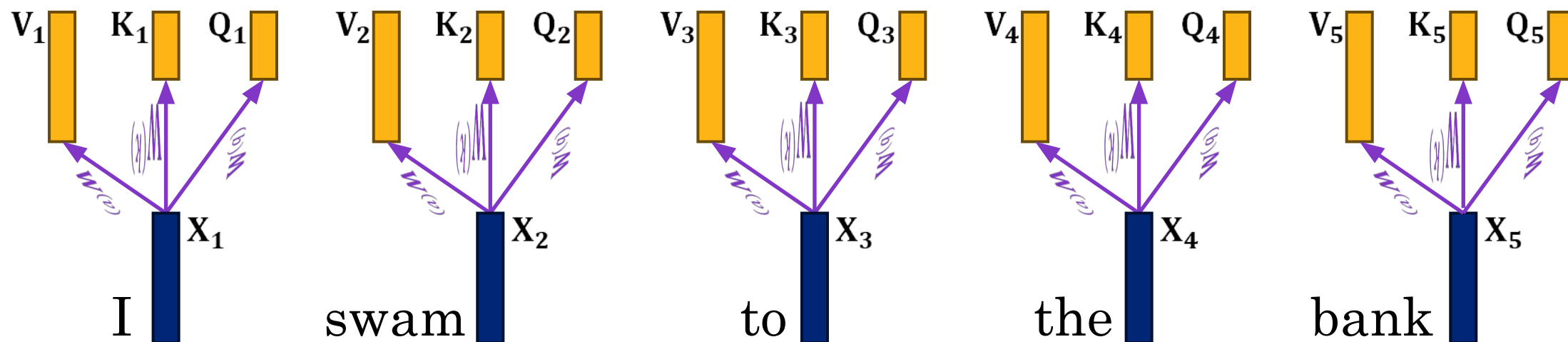
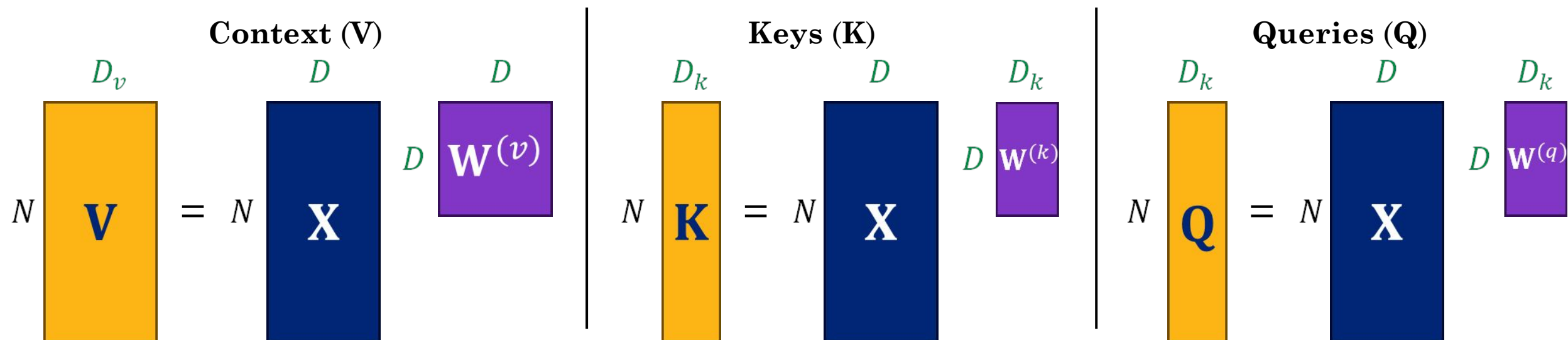
$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$



Matrix Formulation of K, Q, V



2761902





Why A Separate Key and Query?

Separate **key** and **query** linear layers (i.e., $\mathbf{W}^{(k)}$, $\mathbf{W}^{(q)}$) allow for **asymmetric queries**

Example:

*I swam across the river to get to the other **bank**.*
Most important Most important

Bank can query river but river doesn't have to query bank.

- **Not all relevance relationships are symmetric**

Inner-Product Search for Tokens

Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

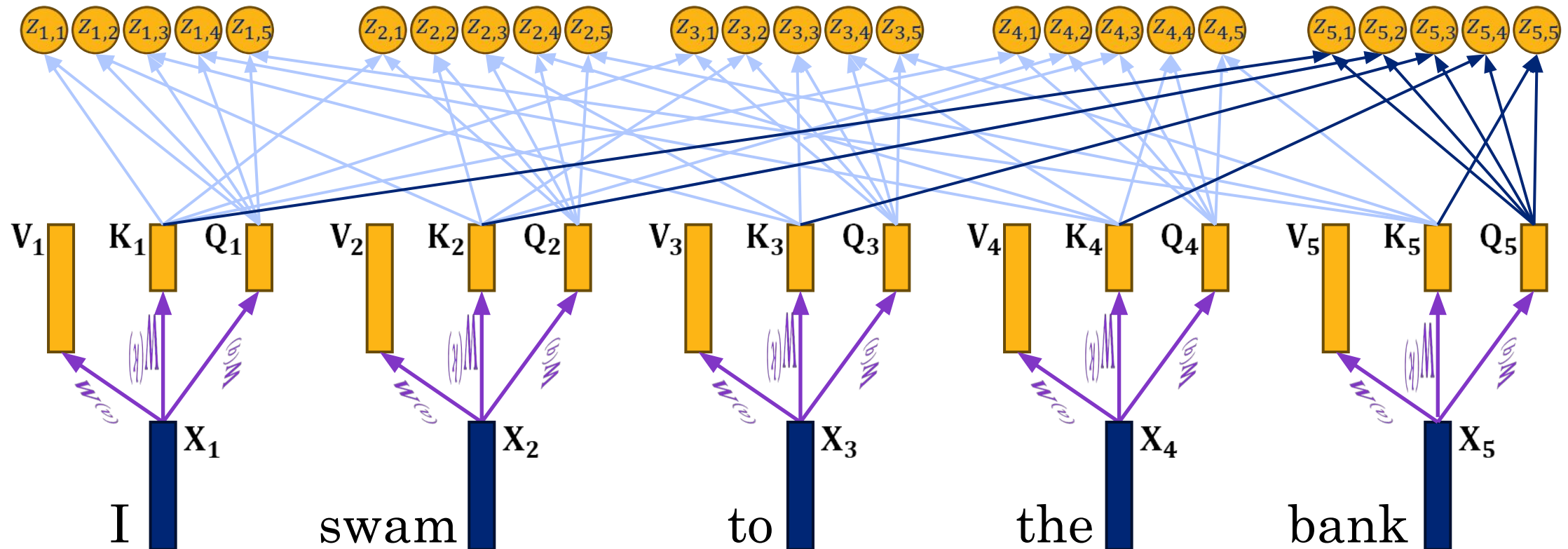
$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$



The **inner-product** between $\mathbf{Q}_n$ and $\mathbf{K}_i$ encodes the **relevance** of **token i** to computing the **representation** of **token n** .



The Inner-Product Matrix

The $\mathbf{Z}_{n,i}$ entry of the $\mathbf{Z} \in \mathbb{R}^{N \times N}$ **inner-product** matrix contains the inner product between query $\mathbf{Q}_n$ and key $\mathbf{K}_i$

Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

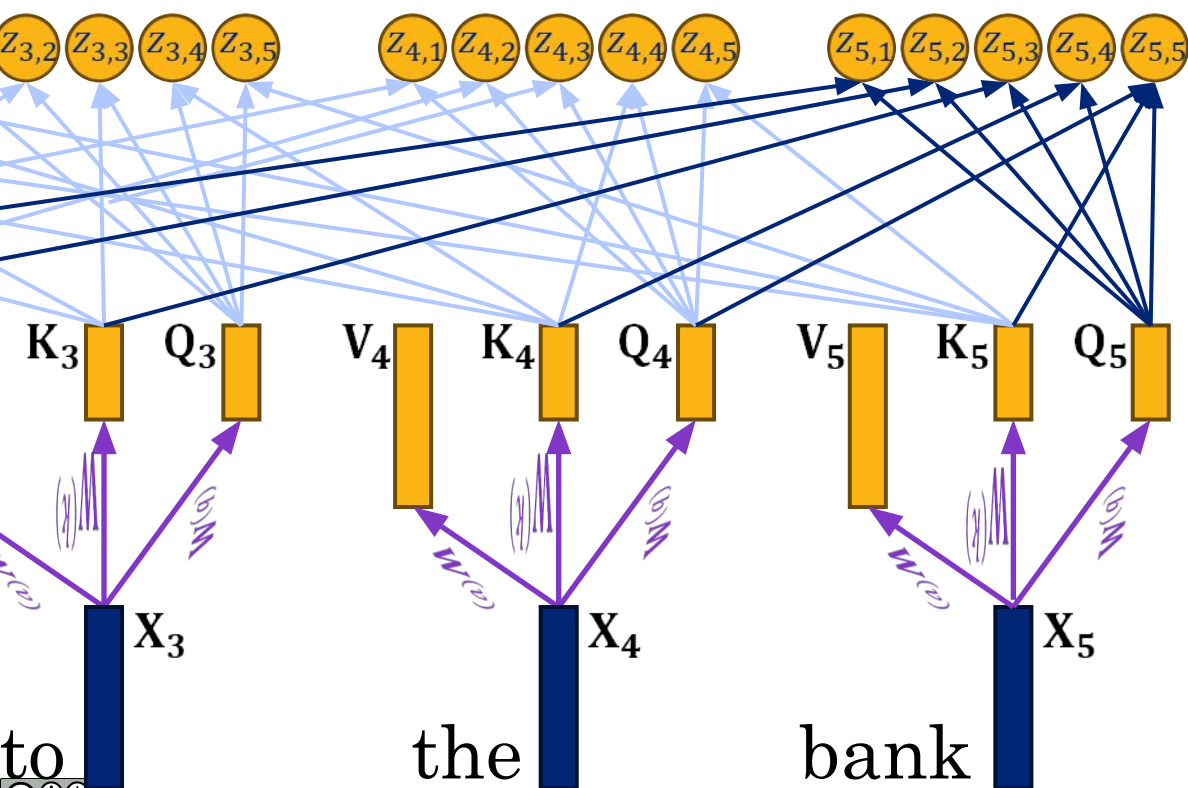
$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$



2761902



$$\mathbf{Z} = \mathbf{Q} \mathbf{K}^T$$

Issue:

$z_{n,i} \in \mathbb{R}$ could be negative and not guaranteed $\sum_{i=1}^N z_{n,i} = 1$

Solution:

Softmax



Softmax Inner-Product Self-Attention

Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

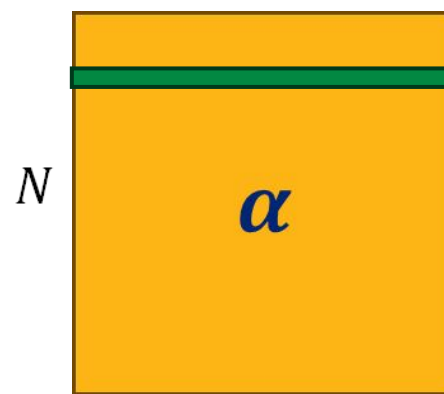
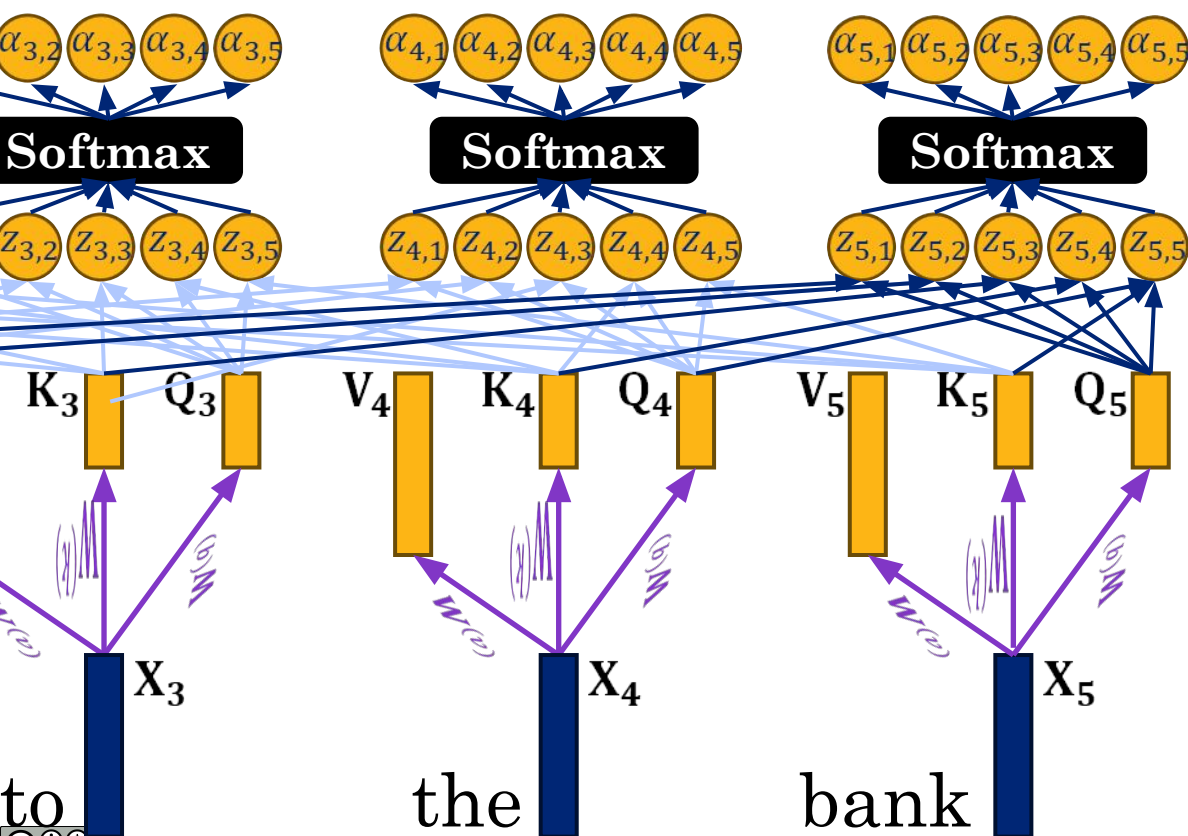
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

By computing the **softmax** along the rows of $\mathbf{Z}$ obtain the **attention matrix** $\alpha \in \mathbb{R}^{N \times N}$

$$\mathbf{Z} = \mathbf{Q} \mathbf{K}^\top$$

$$\alpha = \text{SoftMaxRow}[\mathbf{Z}]$$

$$\alpha_{ni} = \frac{\exp(\mathbf{Z}_{ni})}{\sum_{j=1}^N \exp(\mathbf{Z}_{nj})}$$



Each row has
no negative entries
and sums to 1

Scaled Softmax Inner-Product Self-Attention

Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

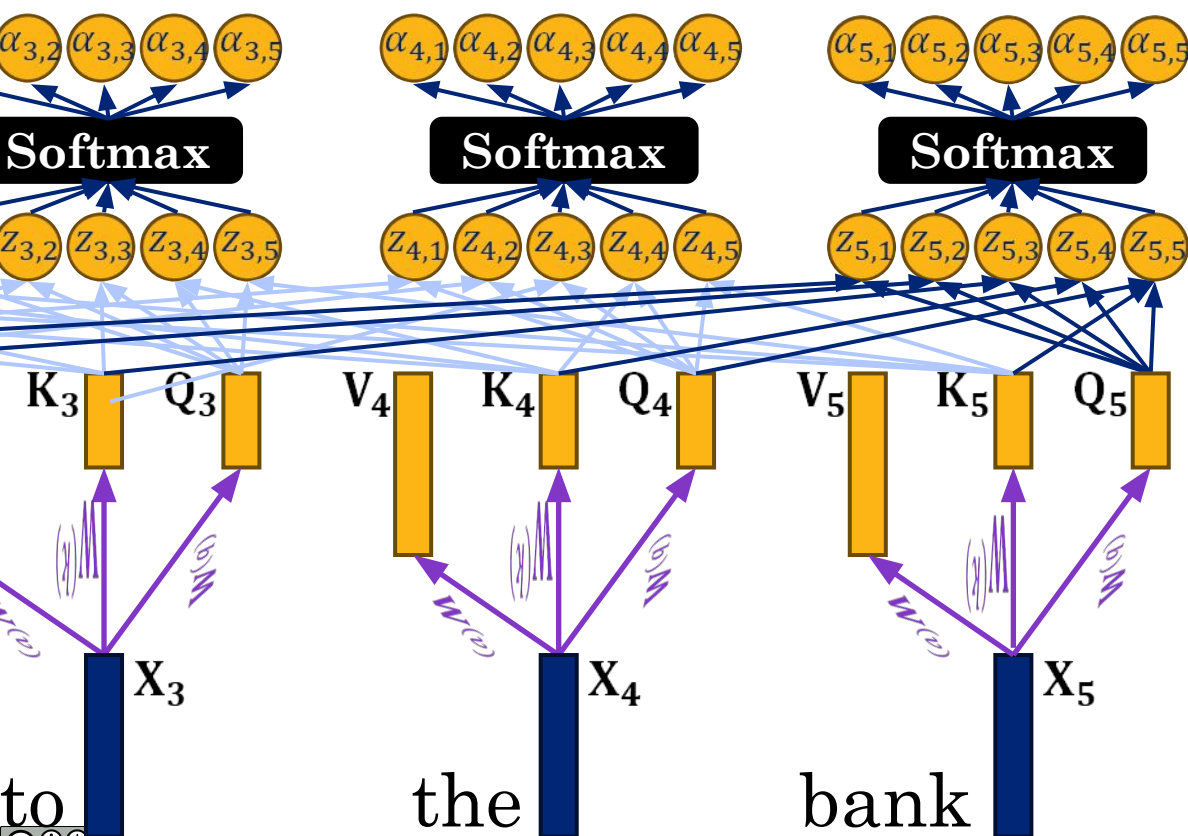
$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$



2761902

By computing the **softmax** along the rows of $\mathbf{Z}$ obtain the **attention matrix** $\alpha \in \mathbb{R}^{N \times N}$



$$\mathbf{Z} = \mathbf{Q} \mathbf{K}^T$$

$$\alpha = \text{SoftMaxRow} \left[\frac{\mathbf{Z}}{\sqrt{D_k}} \right]$$

$$\alpha_{ni} = \exp \left(\frac{\mathbf{Z}_{ni}}{\sqrt{D_k}} \right) \left(\sum_{j=1}^N \exp \left(\frac{\mathbf{Z}_{nj}}{\sqrt{D_k}} \right) \right)^{-1}$$

Scaled by $\sqrt{D_k}$ to ensure that variance of the inner product of D_k terms stays close to 1.

Improves training...

The Attention Layer

$$\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}] = \boldsymbol{\alpha} \mathbf{V} = \sum_{i=1}^N \alpha_{ni} \mathbf{V}_i = \sum_{i=1}^N \left(\text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

Symbols:

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

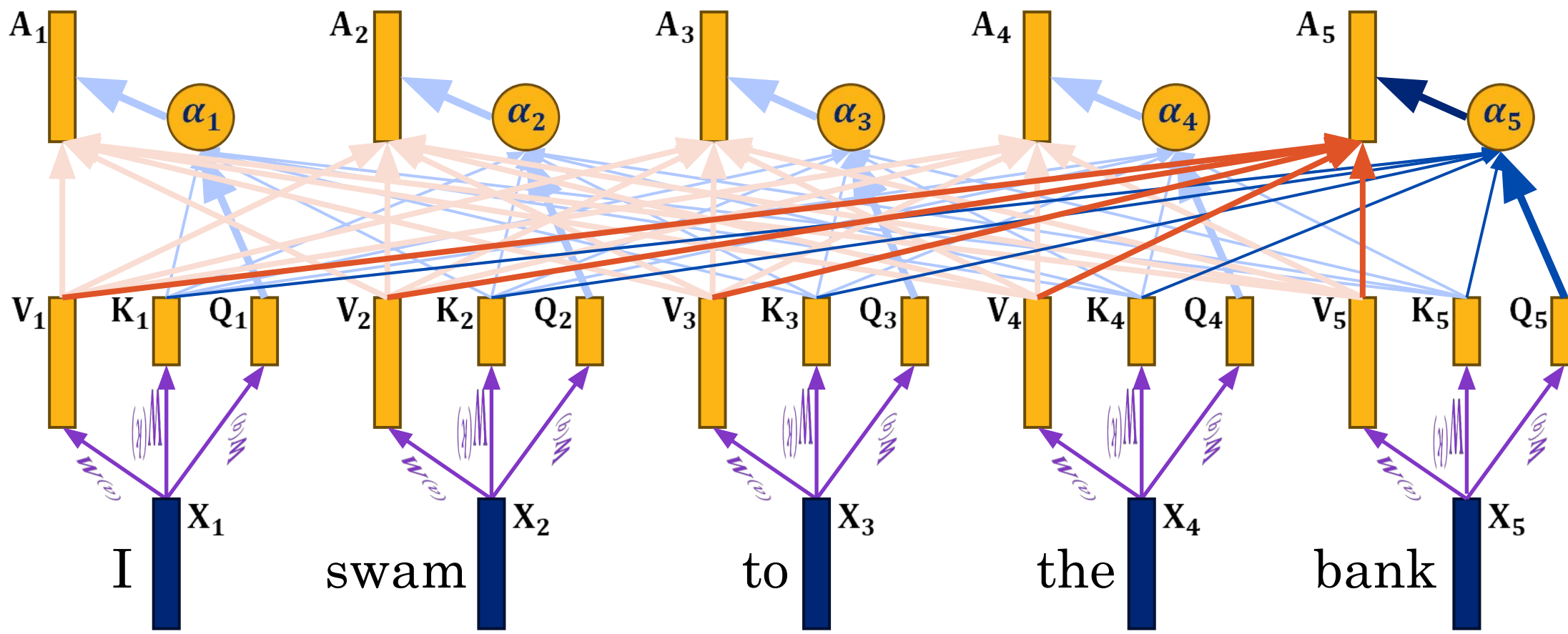
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

$$\boldsymbol{\alpha} = \text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right]$$

$$\boldsymbol{\alpha} \in \mathbb{R}^{N \times N}$$



2761902



The Attention Layer

$$\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}] = \boldsymbol{\alpha} \mathbf{V} = \sum_{i=1}^N \alpha_{ni} \mathbf{V}_i = \sum_{i=1}^N \left(\text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

Symbols:

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

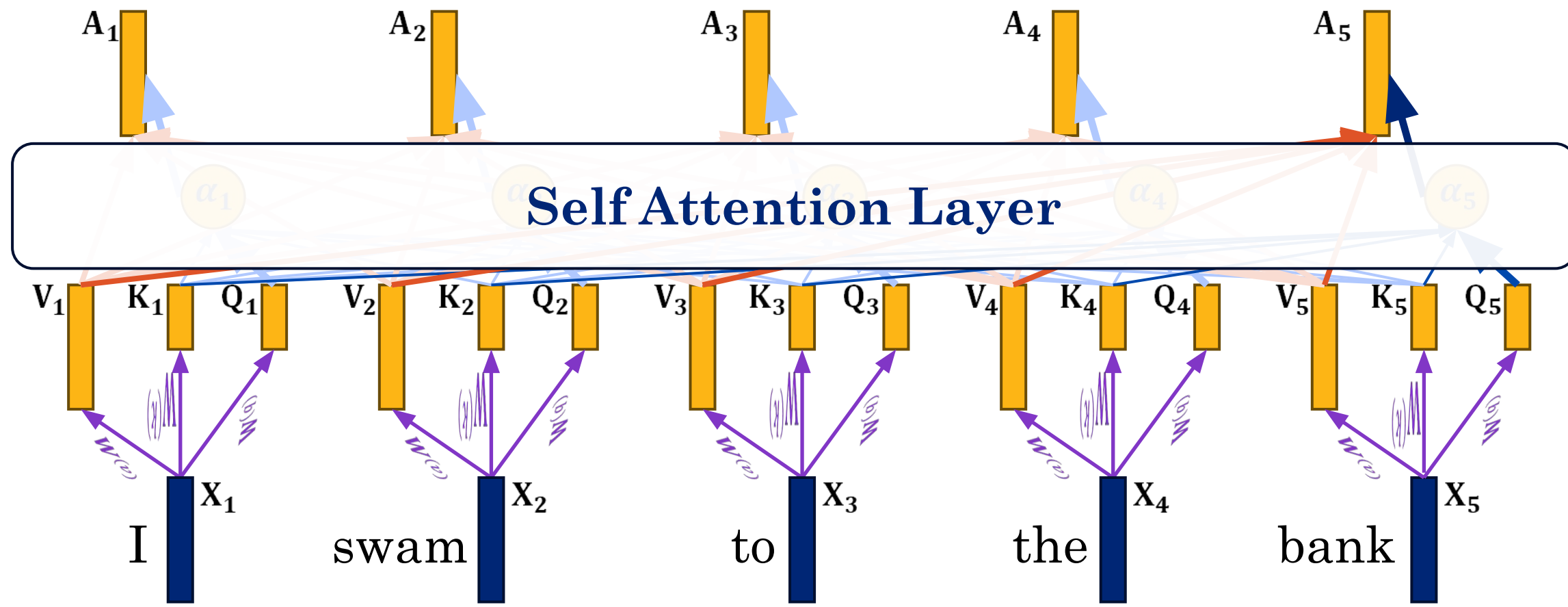
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

$$\boldsymbol{\alpha} = \text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right]$$

$$\boldsymbol{\alpha} \in \mathbb{R}^{N \times N}$$



2761902



The Self Attention Layer

The attention layer $\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]$ depends only on keys $\mathbf{K}$, queries $\mathbf{Q}$, and values $\mathbf{V}$ for all tokens and has no parameters.

Symbols:

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

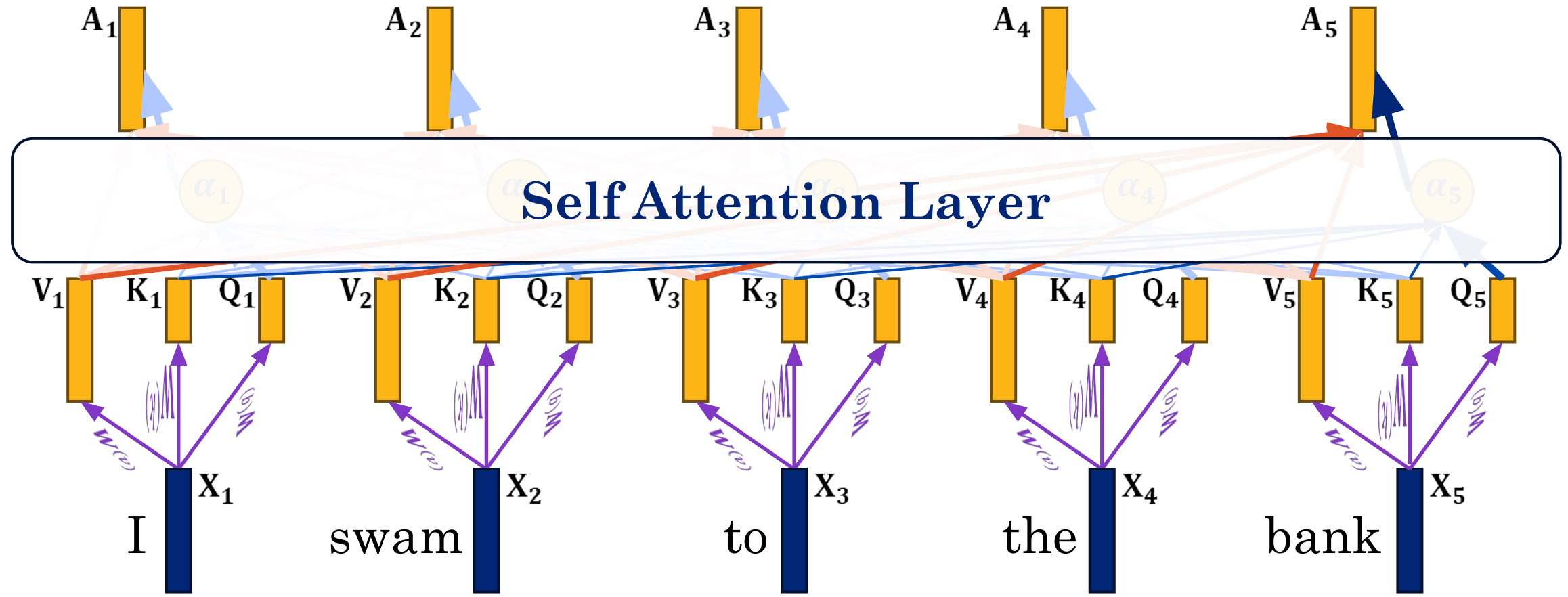
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

$$\alpha = \text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right]$$

$$\alpha \in \mathbb{R}^{N \times N}$$



2761902





Parameter and Computational Complexity

How many parameters are needed to construct $\mathbf{K}$, $\mathbf{Q}$, and $\mathbf{V}$?

- Add the parameters $D_v^2 + 2D_k^2$ which **doesn't depend** on N

What is the **computational complexity** of $\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]$?

- $\text{Softmax}\left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}}\right]$ is $O(N^2 D_k)$
- $\alpha \mathbf{V}$ is $O(N^2 D_v)$ ← Dominant term

Compared to a **Fully Connected Layer**

- The **flattened feature** vector would be in $\mathbb{R}^{N D_v}$
- Linear layer parameters: $(N D_v)^2 = N^2 D_v^2$
- **Computation:** $O((N D_v)^2)$

Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}, \mathbf{W}^{(v)} \in \mathbb{R}^{D_v \times D_v}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}, \mathbf{W}^{(k)} \in \mathbb{R}^{D_k \times D_k}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}, \mathbf{W}^{(q)} \in \mathbb{R}^{D_k \times D_k}$$

$$\alpha = \text{Softmax}\left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}}\right], \alpha \in \mathbb{R}^{N \times N}$$

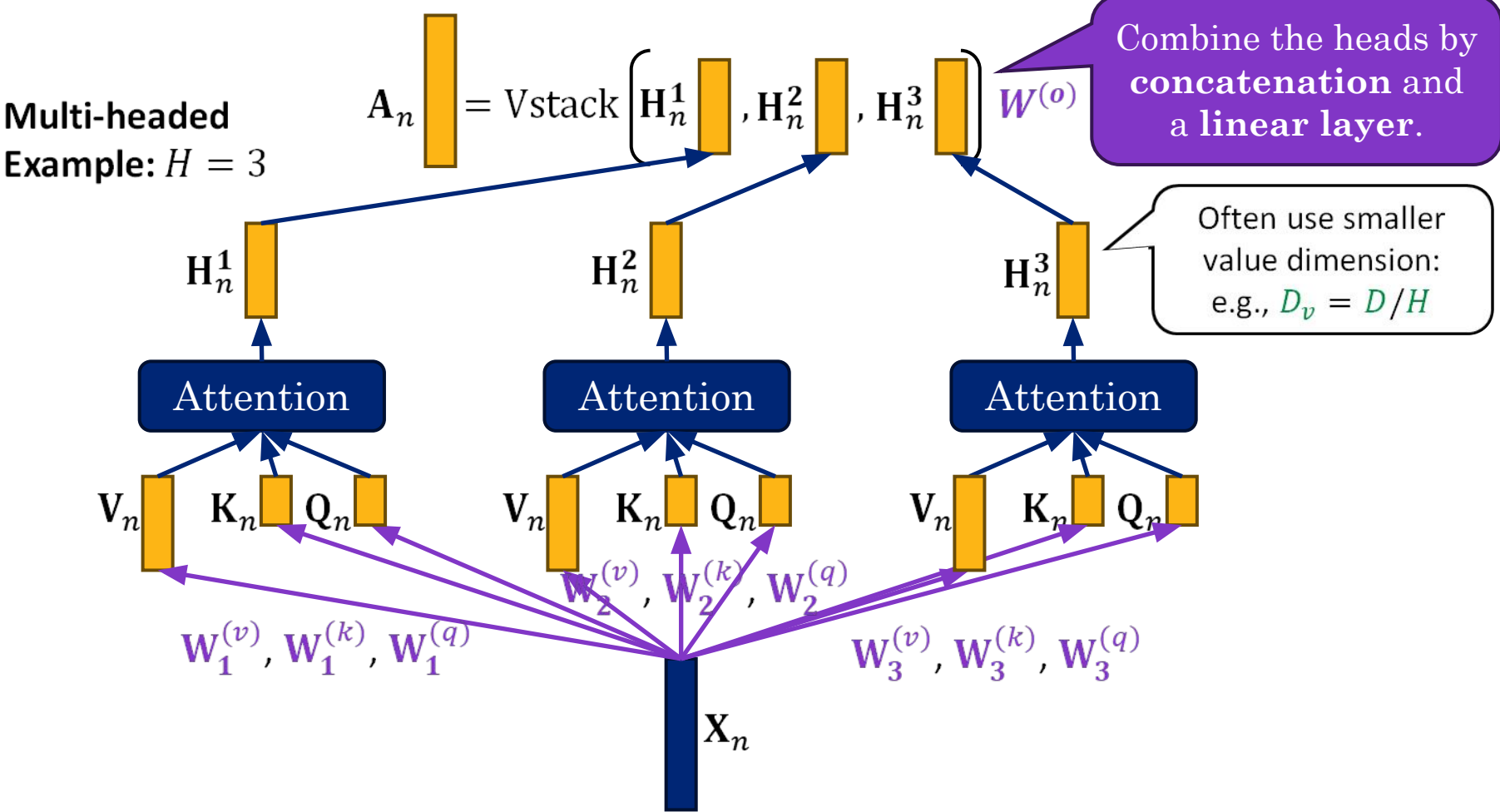
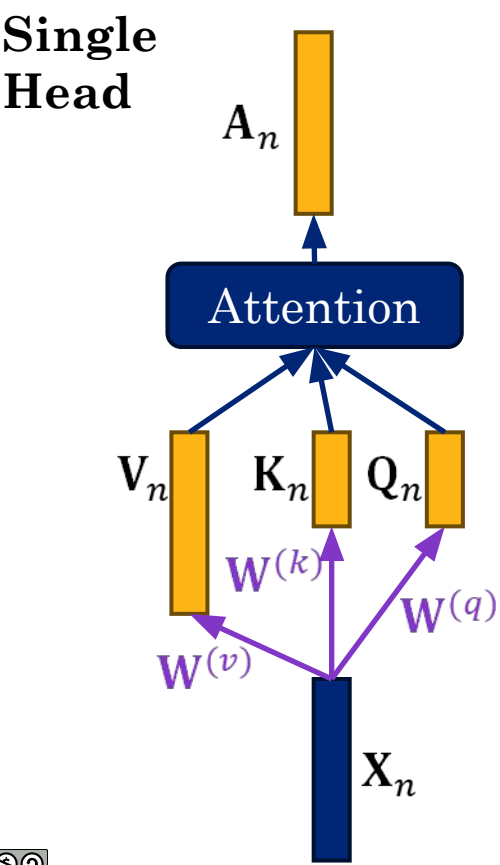
$$\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}] = \alpha \mathbf{V}$$



Multi-Headed Attention

Idea: There may be more than one way to **extract** and **query** the context.

Multi-headed attention makes H parallel copies of the **K**, **Q**, and **V** matrices with **separate weights**.

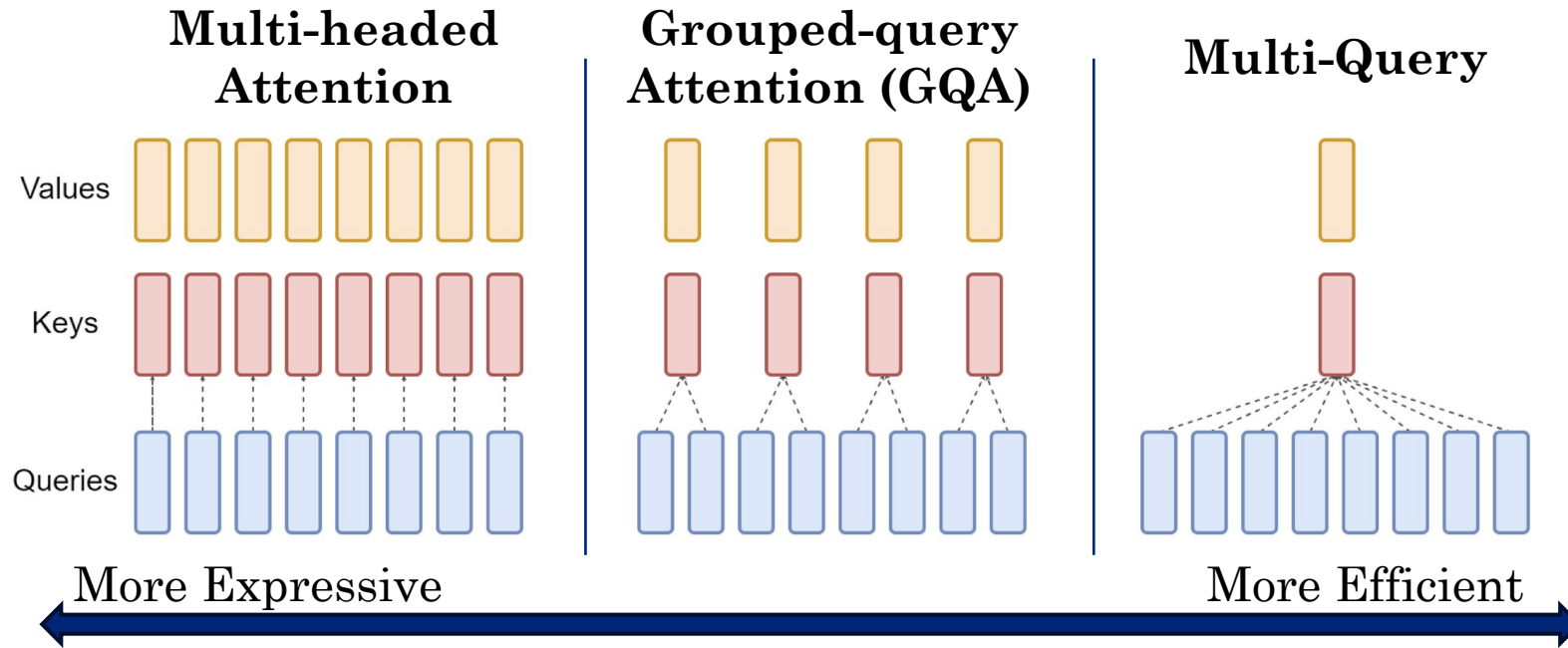




Alternatives to Multi-Headed Attention

Creating (and storing) keys $\mathbf{K}$, queries $\mathbf{Q}$, and values $\mathbf{V}$ for each head is costly.

- Could we reuse the same **keys** and **values** with **multiple queries**?



Most models currently use **Grouped Query Attention (GQA)** which reuses **keys** and **values** across multiple **query** vectors.

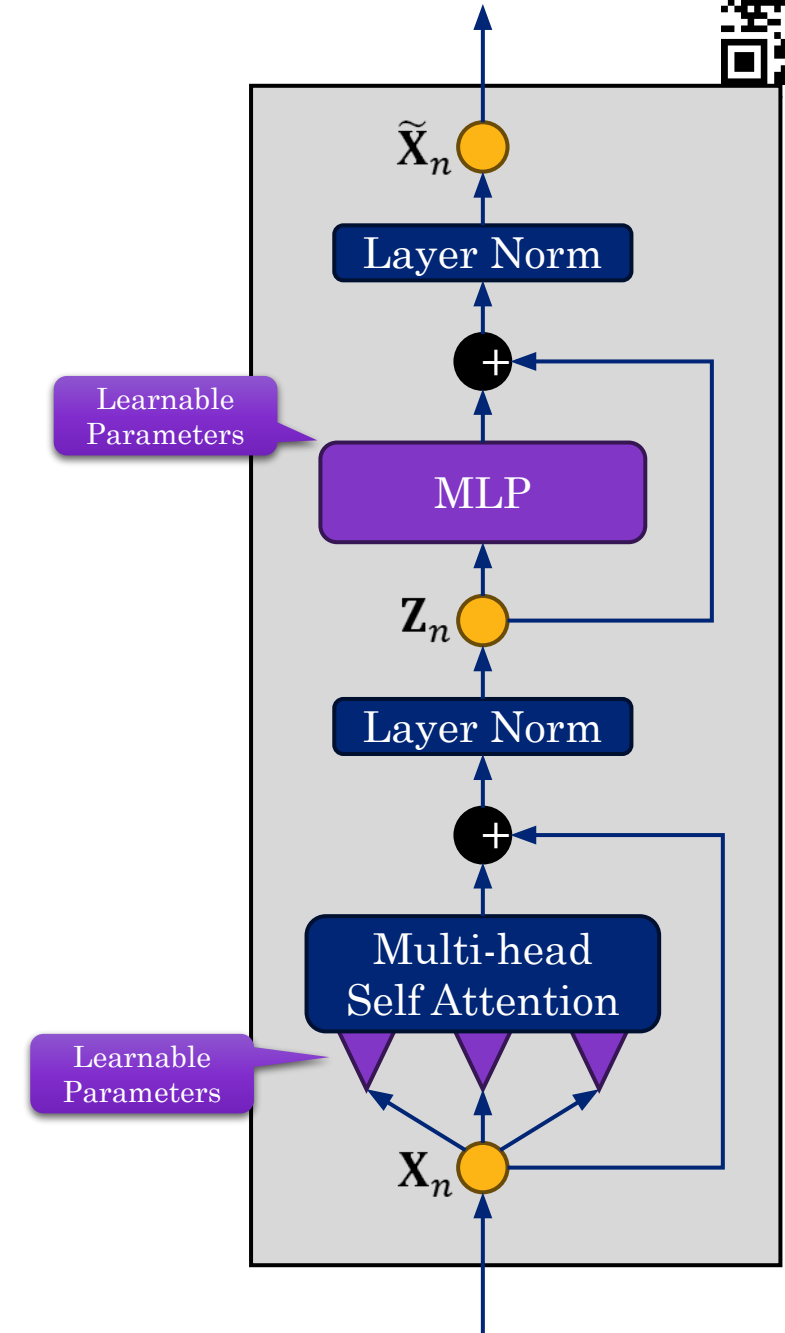
The Transformer Layer

A **transformer layer** combines **self-attention** with a token-level **neural network** (typically, a 2-layer MLP).

- Why is the the MLP is needed?
 - The output of attention is a **linear combination** (αV) of **linear transformations** ($V = X W^{(v)}$) on the input though α is a **non-linear function** of X .

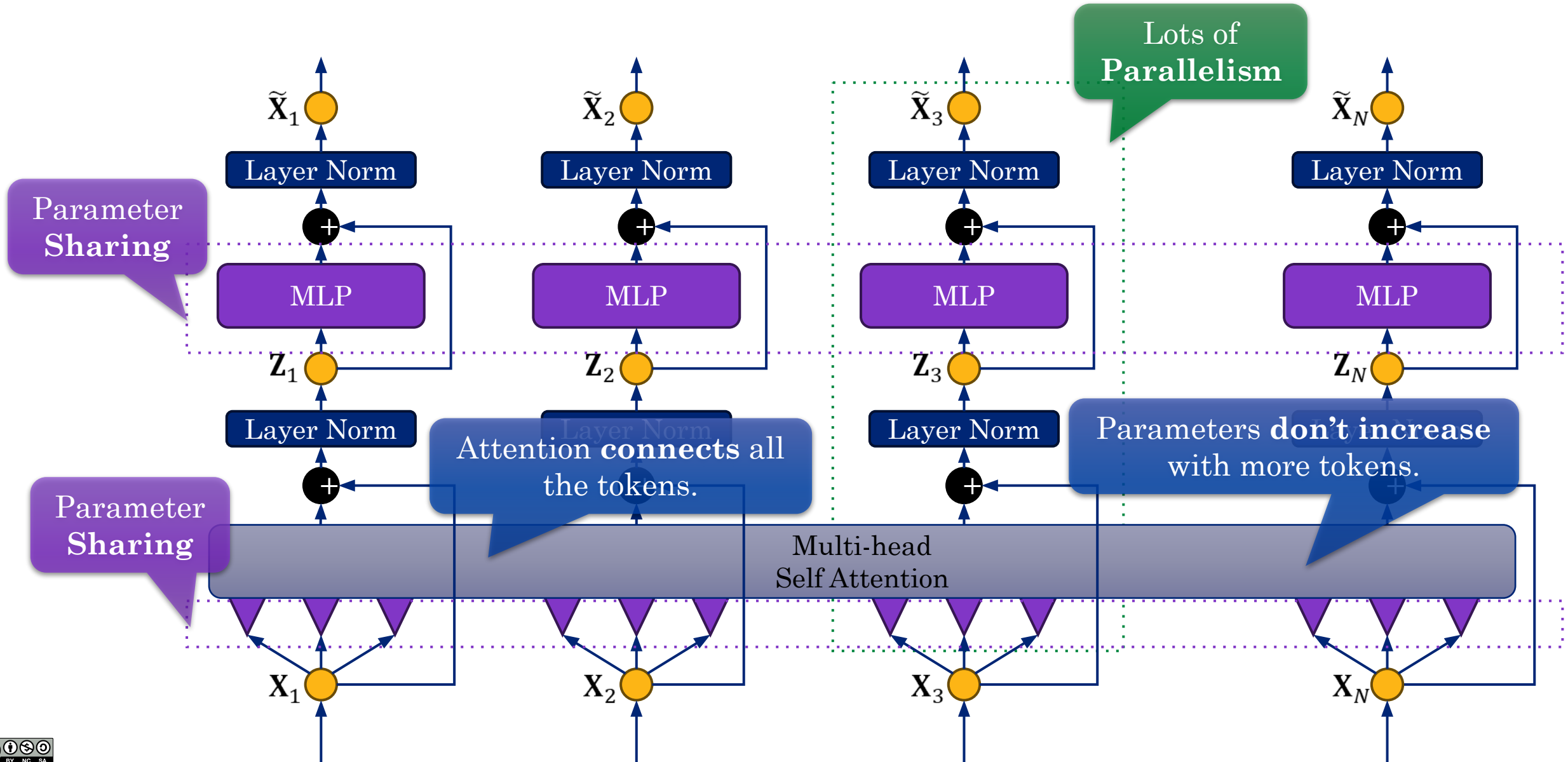
Residual connections and **layer norm** are used to improve training

Transformer layers are repeated **many times** in a transformer model.



61902

Unrolling the Transformer Layer

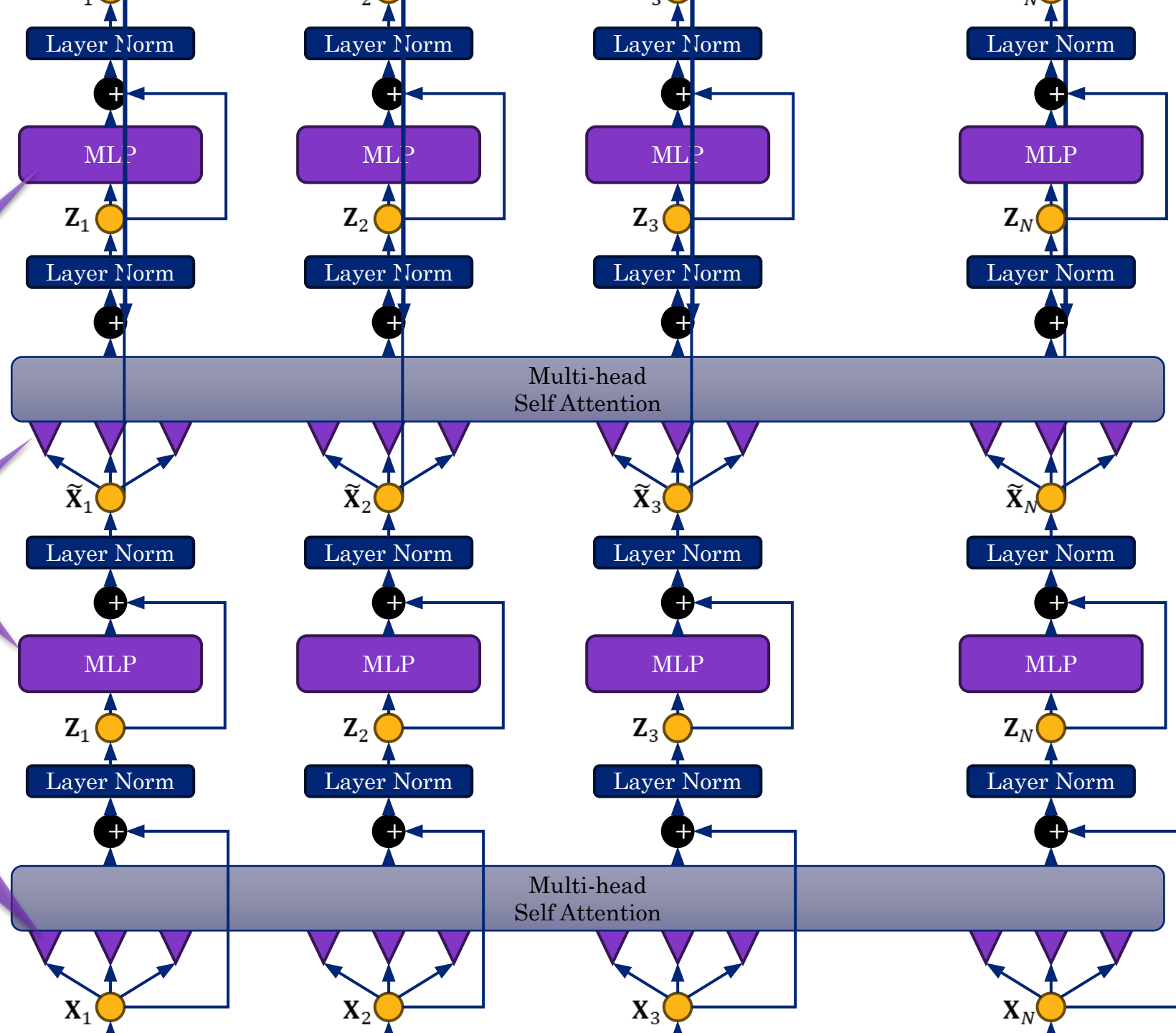


Stacking Transformer Layers

Different Weights!
(not shared)

Different Weights!
(not shared)

Each layer has
new weights.



Reading the Output of the Transformer



Suppose we want to apply the **transformer model** architecture to **image classification**.



Reading the Output of the *former*



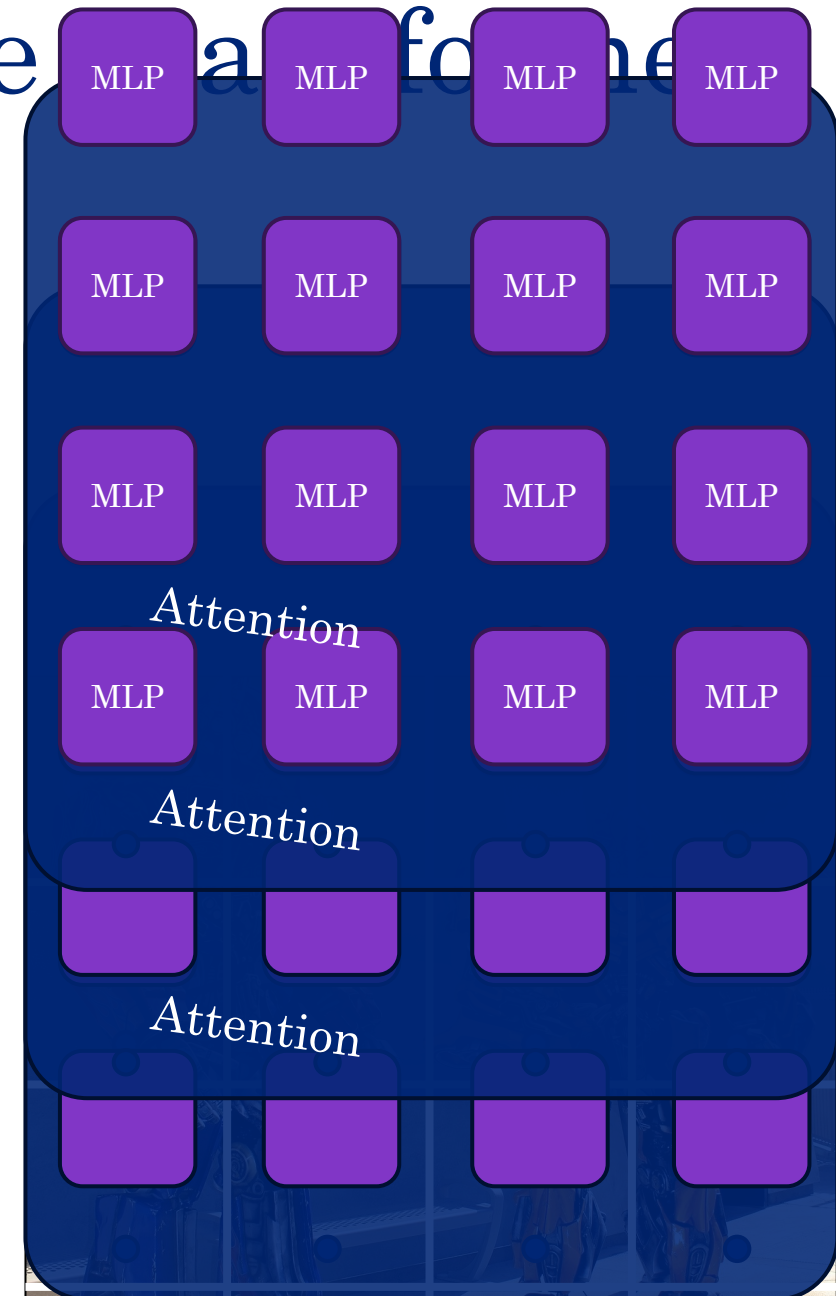
Suppose we want to apply the **transformer model** architecture to **image classification**.

The top layer now has many MLP outputs.

How do we combine them?

- Pooling + Linear Layer
- Concatenation + Layer

Common Trick:



Reading the Output of the Transformer

Suppose we want to apply the **transformer model** architecture to **image classification**.

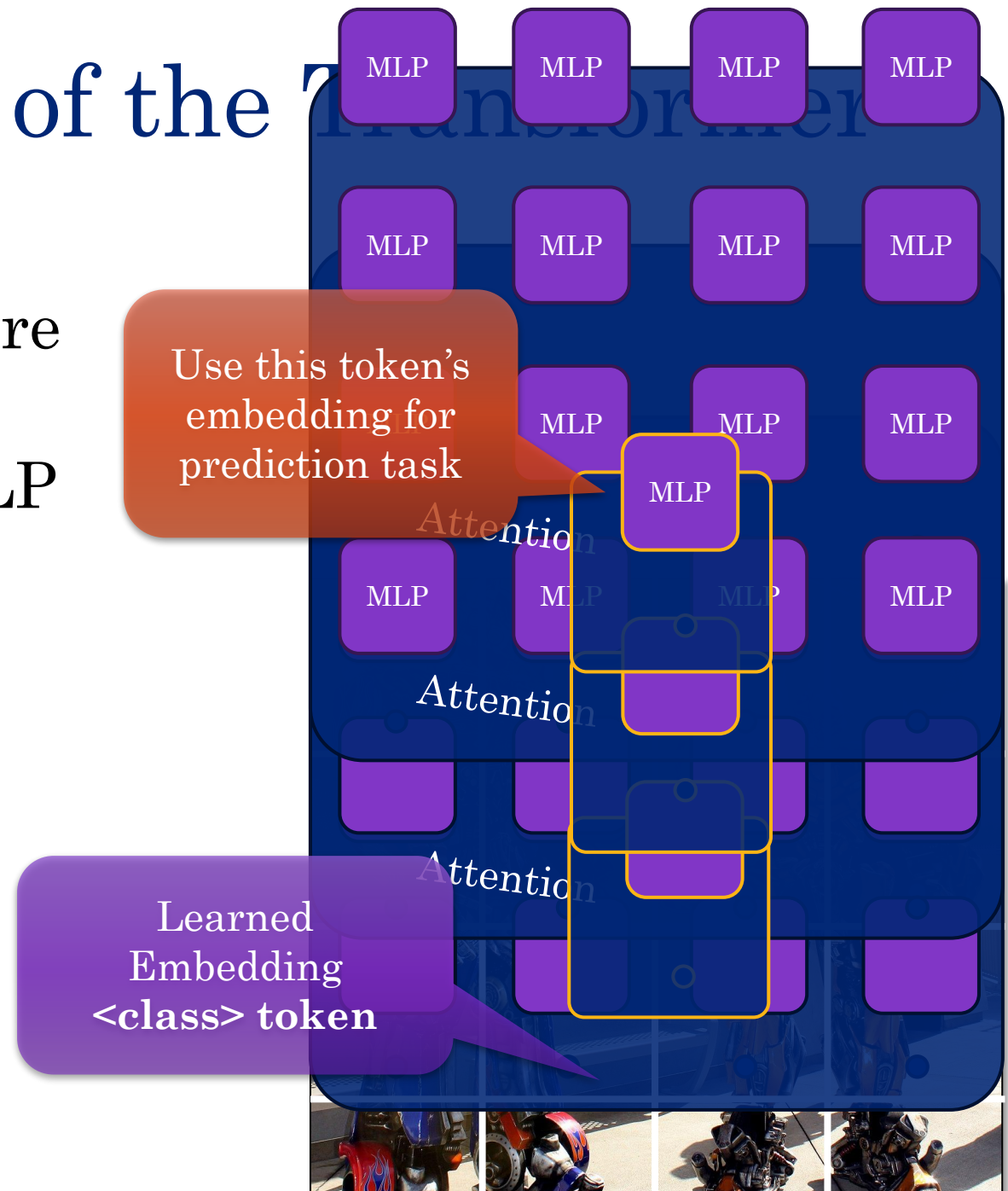
The top layer now has many MLP outputs.

How do we combine them?

- Pooling + Linear Layer
- Concatenation + Layer

Common Trick:

- **Add another token**



2761902

Stopped Here

We will review visual transformers and continue from here next lecture.



Transformers and Token Ordering

Does the **order of tokens matter** in the transformer model?

If I **rearrange the tokens**, would I get a different output for **each token**?

Recall the attention operation is:

$$\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]_n = \sum_{i=1}^N \alpha_{ni} \mathbf{V}_i = \sum_{i=1}^N \left(\text{Softmax} \left[\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

the **weighted averaging** which is **order invariant**.

- Attention is **equivariant** – permuting the input tokens permutes the outputs.

Is this a problem?

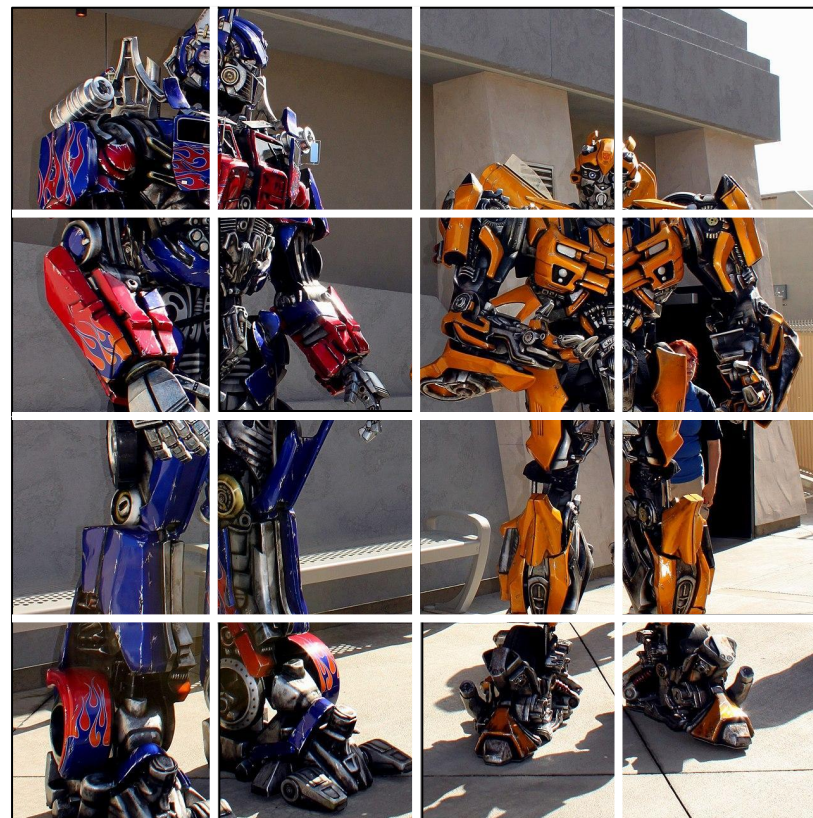
- **No.** If you want a neural network for **sets of objects**.
- **Yes!** If you want to model **images** or **text**.



Token Position in Images and Text



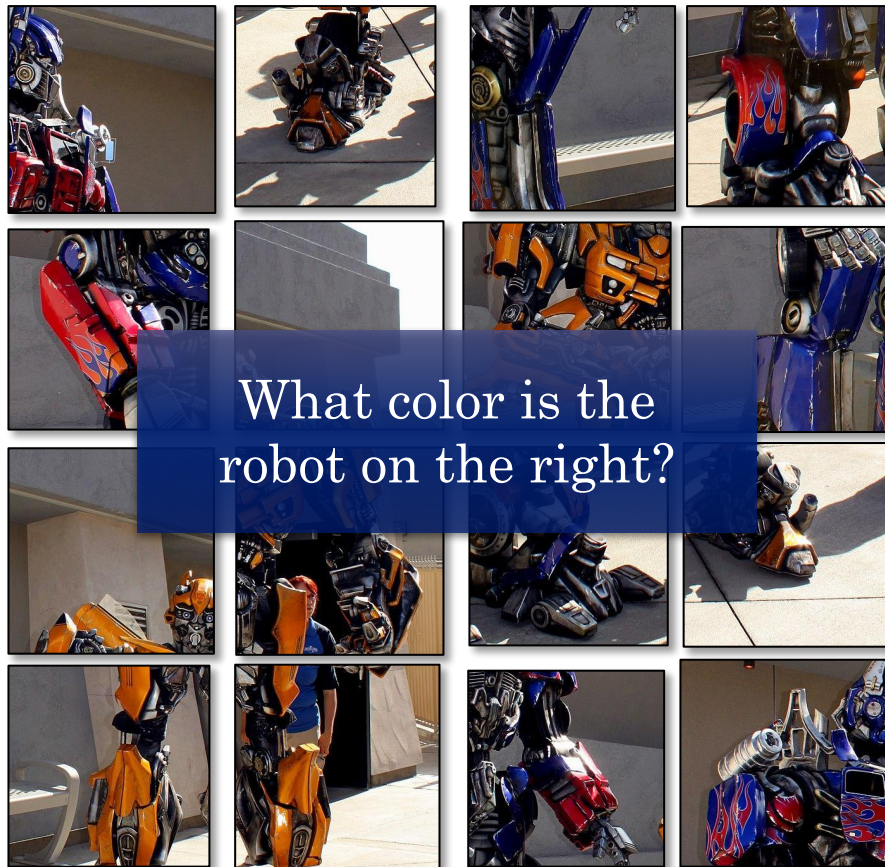
Positional information is important for understanding images and text.



Token Position in Images and Text



Positional information is important for understanding images and text.

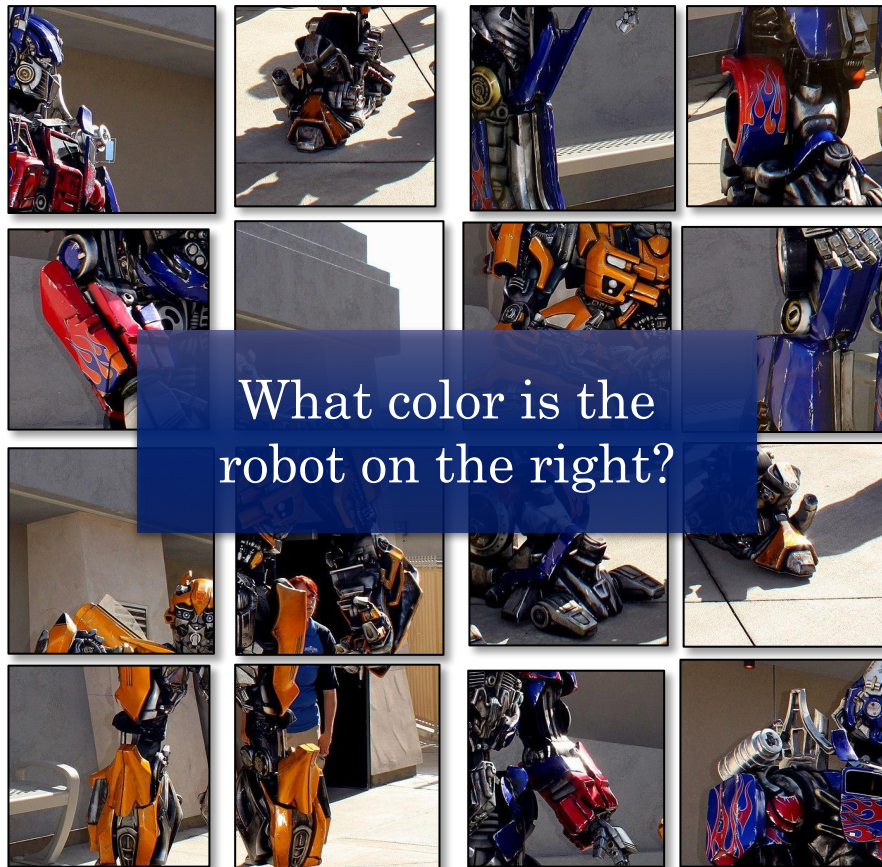


The	food	was	good	not	bad	at	all!
-----	------	-----	------	-----	-----	----	------

Token Position in Images and Text



Positional information is important for understanding images and text.



The	food	was	good	not	bad	at	all!
-----	------	-----	------	-----	-----	----	------

The	food	was	bad	not	good	at	all!
-----	------	-----	-----	-----	------	----	------

A small permutation in the order changes the meaning!



Positional Encoding

Modify the raw features to contain information about the position of the token in the input. (**Feature Engineering**)

We add a **positional embedding** $\mathbf{r}_n \in \mathbb{R}^D$ **vector** encoding the position to the initial input embedding of each token $\tilde{\mathbf{x}}_n$:

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$$

- Why not **vstack** $[\mathbf{x}_n, \mathbf{r}_n]$?
 - Increases the dimensionality changing the model.
- Wouldn't this corrupt $\mathbf{x}_n$?
 - For high-dimensional vectors $\mathbf{x}_n$ and $\mathbf{r}_n$ are likely to be almost orthogonal.
 - Linear layers would end up adding $\mathbf{x}_n$ and $\mathbf{r}_n$ if we had used **vstack** $[\mathbf{x}_n, \mathbf{r}_n]$
- How do we construct $\mathbf{r}_n$?



Designing the Position Embedding

An ideal positional encoding should:

1. Provide a **unique representation** $\mathbf{r}_n \in \mathbb{R}^D$ for each location.
2. Be **bounded** on the scale of the original features $\mathbf{x}_n$ (e.g., $[-1,1]$).
 - Needed for **stable training**.
3. Easily express **relative distances** between tokens
 - **Inductive Bias:** relative location is important for images and language.
4. Should work for **arbitrary number of tokens** N
 - Not all sentence or images are the same size.



Learned Positional Embeddings

The first GPT model (GPT-1) used **learned positional embeddings** in which each **positional embedding vector** $\mathbf{r}_n \in \mathbb{R}^D$ is learned through gradient descent.

- The vectors $\mathbf{r}_n$ were randomly initialized (small values)

Strengths:

- Expressive and easy to implement

Weaknesses:

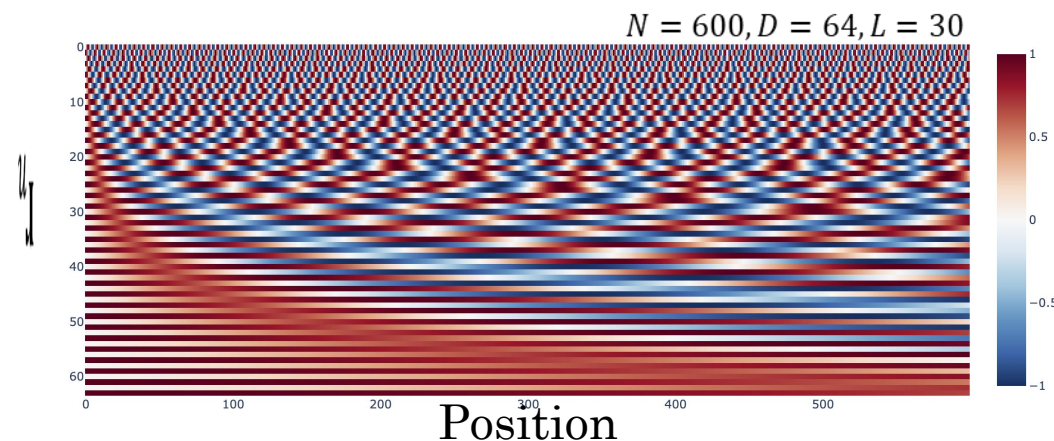
- The maximum N needs to be known in advances
- Relative distances can't be easily computed (needs to be learned)

Sinusoidal Positional Encoding



The original Transformer paper used sinusoidal embeddings to encode positions through a combination of waves.

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd} \end{cases}$$



Like a “binary” encoding of the position but continuous:

Position	$\mathbf{r}_3$															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
2	0	0	0	1	1	1	0	0	0	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
4	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1



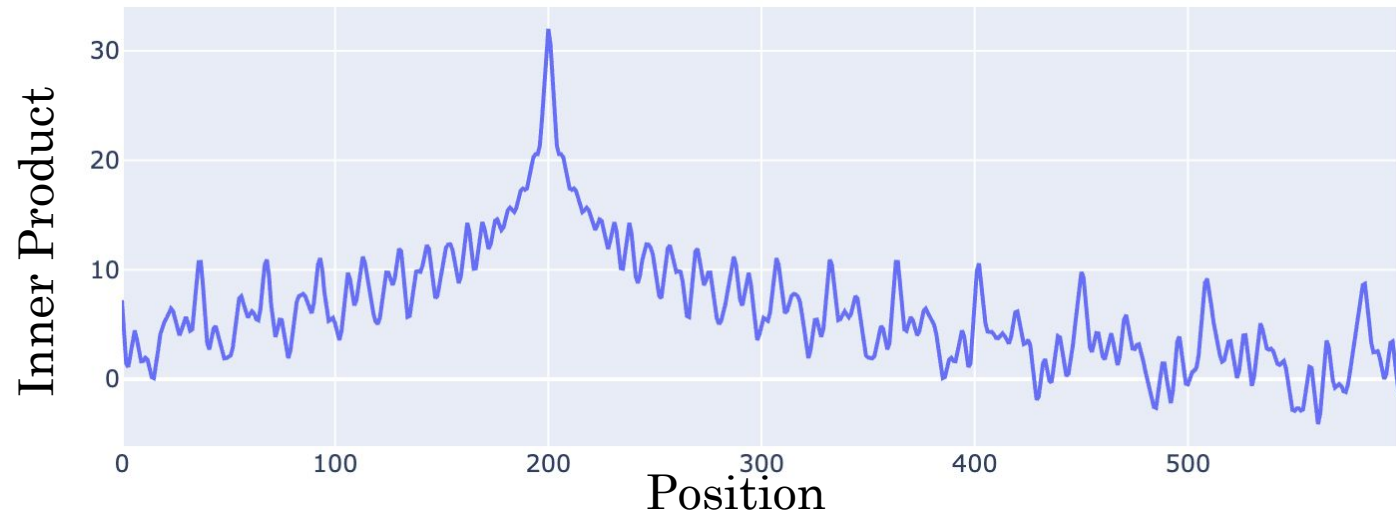
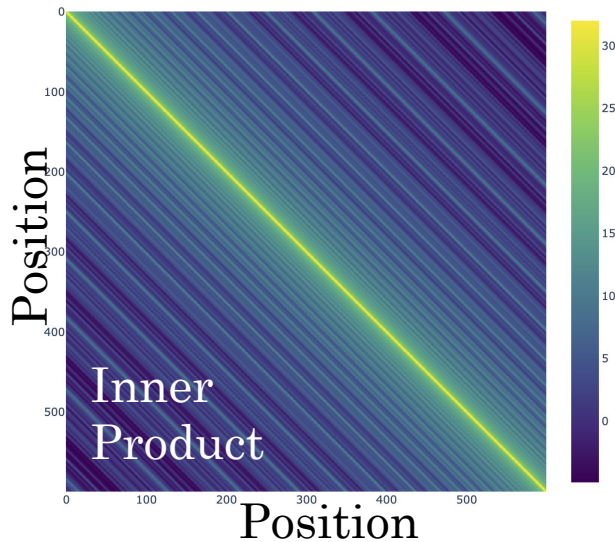
Querying a Relative Position

The sinusoidal position enables **linear layers** to query a fixed offset.

Suppose I have $\mathbf{r}_n$ and I want to query a key at $\mathbf{r}_{n-\Delta}$ then there exists a rotation matrix $\mathbf{R}_\Delta$ such that:

$$\mathbf{r}_{n-\Delta} = \mathbf{R}_\Delta \mathbf{r}_n$$

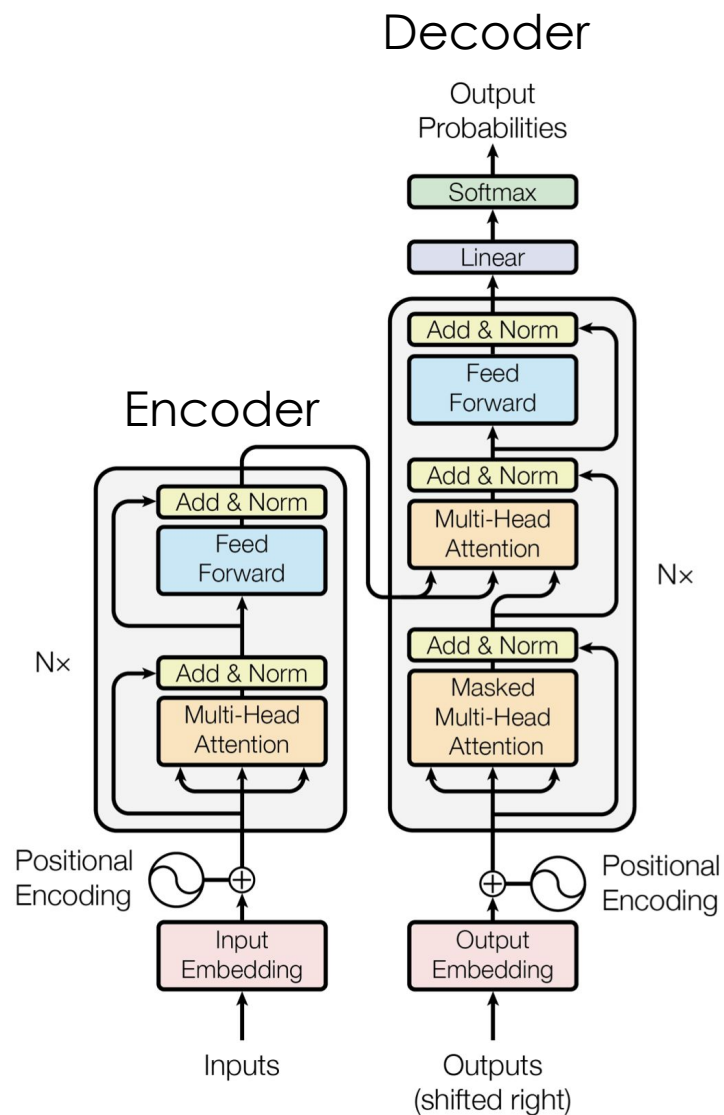
Inner-products of these encodings decrease with distance:



The Encoder Transformer



2761902



arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

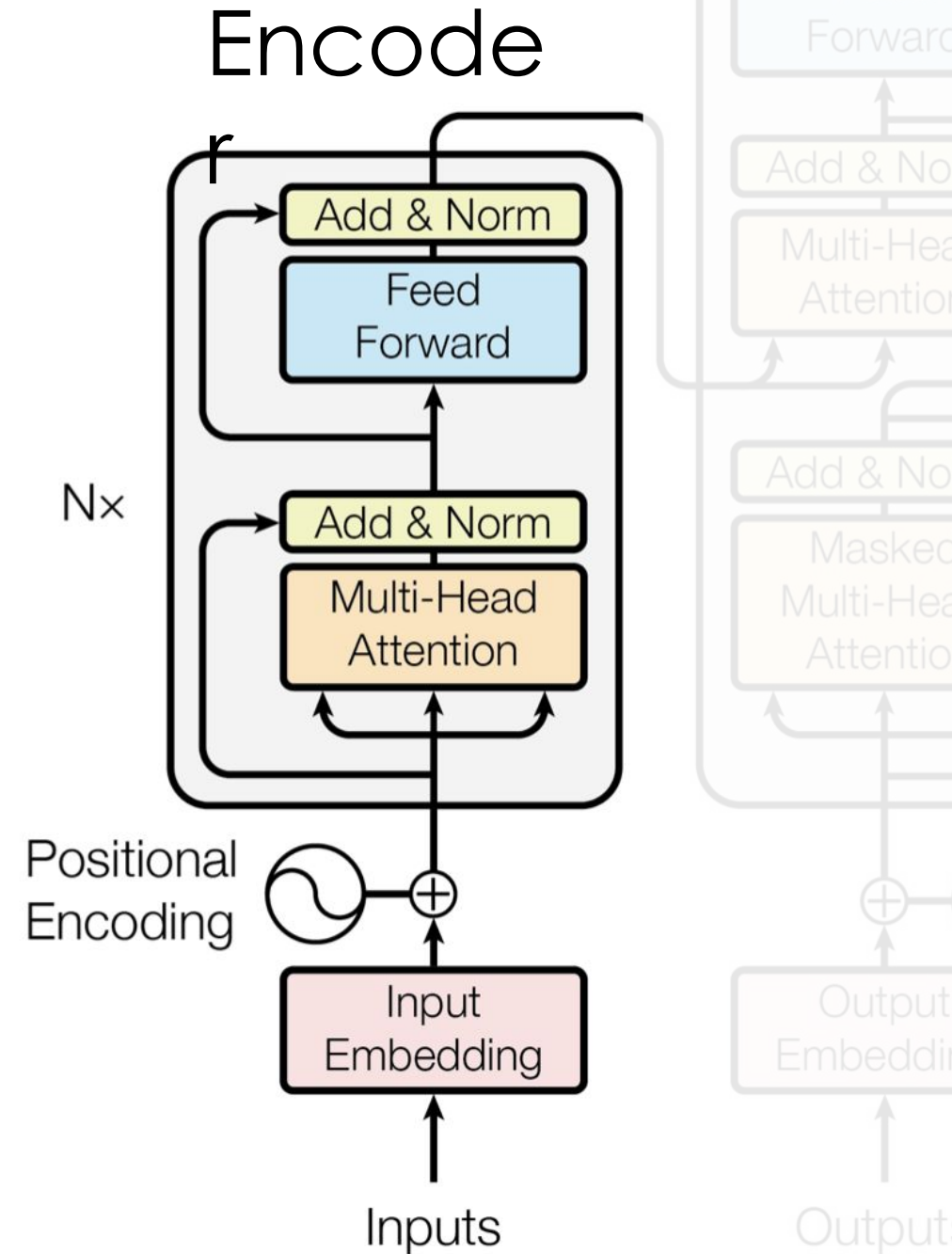
[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

The Encoder Transformer

The encoder transformer architecture is standard for **computer vision tasks** and **language embedding** tasks.

- Construct input embeddings
 - **Example:** Pixel patches
- Apply Positional Encoding
 - **Example:** Sinusoidal encoding
- Many repeated transformer blocks
- Use final output in downstream tasks
 - **Example:** Pooling or Special token



Lecture 20

Transformers

Reading: Chapter 12 in the Bishop Deep Learning Textbook